



AST-guided transformers for sustainable and reliable UML modeling

Feisal Alaswad ^a, Poovammal E. ^a, Batoul Aljaddouh ^a, Kadiyala Ramana ^b,
Arfat Ahmad Khan ^c,*, Natalia Kryvinska ^{d,*}

^a Department of Computing Technologies, SRM Institute of Science and Technology, Kattankulathur, Chengalpattu, 603203, Tamil Nadu, India

^b Department of Artificial Intelligence and Data Science, Chaitanya Bharathi Institute of Technology, Hyderabad, 500075, Telangana, India

^c Department of Computer Science, College of Computing, Khon Kaen University, Khon Kaen, 40002, Thailand

^d Department of Information Management and Business Systems, Faculty of Management, Comenius University Bratislava, Odbojárov 10, P.O. Box 95, 82018 Bratislava 218, Slovakia

ARTICLE INFO

Dataset link: <https://github.com/FeisalAlaswad/GenClass-Seq2AST>, <https://github.com/FeisalAlaswad/PlantUCD-dataset-full>

Keywords:

Model-driven development
Class diagrams generation
Requirement analysis
NLP transformer

ABSTRACT

The manual conversion of natural language software requirements into Unified Modeling Language (UML) class diagrams is often labor-intensive, dependent on expert knowledge, and susceptible to human error within the requirements engineering process. Inefficiencies and inconsistencies introduced during this early design phase can lead to expensive revisions later in development, ultimately affecting the sustainability and long-term maintainability of software systems. To address this challenge, this study presents a neural-based framework that formulates UML diagram generation as a structured machine translation task, aiming to support more reliable and sustainable model-driven engineering practices.

In this work, two transformer-based approaches are proposed: a Sequence-to-Sequence (Seq2Seq) model designed for direct diagram code generation, and a Sequence-to-Abstract-Syntax-Tree (Seq2AST) model that incorporates syntactic constraints to ensure structural correctness while maintaining the intended semantics. To mitigate the limited availability of annotated training data, a multi-task learning strategy is introduced, which simultaneously performs UML element extraction and context-aware recommendation of reusable design patterns derived from a large repository of existing diagrams. Integrating structural constraints with reusable design knowledge enhances modeling consistency, promotes reusability, and supports improved long-term design quality.

Experimental results indicate that the proposed framework achieves accuracy comparable to general-purpose large language models while offering greater determinism and stronger domain alignment. Furthermore, it operates with significantly lower computational requirements, contributing to a more energy-efficient and sustainable integration of artificial intelligence within software engineering workflows.

To support transparency, reproducibility, and continued research in AI-driven systems engineering, all datasets and experimental artifacts used in this study have been made publicly available.

1. Introduction

The manual creation of Unified modeling language (UML) class diagrams from natural language specifications is a labor-intensive and expert-dependent task within software development. Software requirements, frequently documented in natural language, are often characterized by ambiguity, uncertainty, incompleteness, inconsistency, and incoherence. These textual ambiguities and unstructured formats can lead to significant misunderstandings and inconsistencies during the initial stages of software development (Ahmed et al., 2022).

Software engineering, as a sociotechnical discipline, follows a structured life cycle from requirements gathering to delivery (Gerosa et al.,

2024). Machine learning (ML) and deep learning (DL) can enhance the early stages, particularly by analyzing user data and documents to better understand requirements (Budake et al., 2023). Natural language processing (NLP) helps detect patterns and extract relevant information, improving insights into user needs. During the design phase, ML techniques can automate diagram generation (Abdelnabi et al., 2021), streamlining a critical step in software development. Since design forms the blueprint for the entire project, any later changes often require revisiting and reanalyzing the initial requirements, making this process both time- and resource-intensive.

* Corresponding authors.

E-mail addresses: fa5788@srmist.edu.in (F. Alaswad), poovamme@srmist.edu.in (Poovammal E.), ba5329@srmist.edu.in (B. Aljaddouh), kramana_aids@cbit.ac.in (K. Ramana), arafatkhan@kku.ac.th (A.A. Khan), natalia.kryvinska@uniba.sk (N. Kryvinska).

<https://doi.org/10.1016/j.mlwa.2026.100940>

Received 30 March 2026; Received in revised form 8 June 2026; Accepted 21 June 2026

Available online 25 June 2026

2666-8270/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

The general problem addressed in this research is the transformation of textual data written in natural language into structured formal representations. This research specifically focuses on the extraction of UML class diagrams from software requirements expressed in natural language. Class diagrams are a critical component of the UML suite, as they provide a foundational structure upon which other diagrams — such as sequence or activity diagrams — are built. As such, they play a central role in the software design process and are often prioritized in early design stages (Nikiforova et al., 2011).

Several inherent challenges can be anticipated when attempting to convert natural language text into diagrammatic representations:

- Natural language is linear and sequential, whereas diagrams are non-linear and convey information from multiple perspectives. This prevents a simple one-to-one transformation.
- Natural language is rich and complex, while diagrams use limited visual conventions. Translating between them risks losing key details and requires careful abstraction beyond standard text translation.

Many existing methods rely on rule-based or statistical approaches, which often lack generalization. While Large language Models (LLMs) can generate plausible UML drafts, their non-determinism and tendency to hallucinate make them unreliable for precise modeling tasks.

These considerations raise important questions. How can neural translation models preserve semantic and structural fidelity when converting text to UML? Second, given the inherent structural differences, is it possible to model diagrams as decomposable components, independent of linear textual descriptions? To explore these questions, this research introduces a neural framework that treats the problem as a structured translation task.

This research establishes a foundational approach for generating UML class diagrams from natural language requirements by formulating the problem as a Neural Machine Translation (NMT) task. By releasing a publicly available dataset and a reproducible methodology, it aims to support and accelerate future work in this domain. This study focuses on the following objectives:

- Formulate the generation of UML class diagrams from software requirements as a machine translation problem and establish standardized benchmarks for software modeling tasks.
- Introduce the GenClass model (Seq2AST), designed to generate class diagrams from natural language input.
- Investigate strategies to optimize the translation from natural language to UML diagrams.
- Evaluate the effectiveness of transformer-based NLP models in the task of diagram generation.

2. Literature review

This research investigates the conversion of natural language into UML class diagrams by framing it as an NMT task, situated within the well-established NLP literature on DL techniques. Li et al. (2021) introduce PalmTree, which applies self-supervised learning to assembly code, offering insights into low-level embeddings that inform broader code modeling tasks. Similarly, transformer-based models have been used for code generation from natural language (Kusupati & Ailavarapu, 2022). Notably, MarianCG (Soliman et al., 2022) repurposes the Marian NMT model to translate natural language into Python code, demonstrating the adaptability of pre-trained translation models, though with limitations in capturing higher-level code semantics.

Translating user requirements into UML diagrams remains a significant challenge, especially when requirements are expressed in lengthy, unstructured natural language. Such inputs often contain ambiguity, informal terminology, or implicit information that complicates their automatic interpretation and modeling (Alaswad et al., 2023; Zhong, Wu,

Li, Peng, & Wu, 2023). The study by Herchi and Abdesslem (2012) introduced a method to simplify deriving class diagrams from textual requirements by employing a heuristic approach. Ontology learning involves the automated extraction of concepts, relationships, characteristics, and hierarchies from textual data (Asim et al., 2018; Gruber, 1995). Zhong, Scarinci, and Cicirello (2023) suggested a method to support systems engineers in automatically creating Systems Modeling Language (SysML) diagrams, which include concepts, relationships, and characteristics from unstructured natural language text.

In this context, two main approaches are available: the semi-automated approach and the fully automated approach. Both aim to simplify the conversion of textual requirements into visual UML representations, but they vary in terms of human participation and the extent of automation (Abdelnabi et al., 2021). RAPID presented by More and Phalnikar (2012), a technique that transforms software requirements into UML class diagrams using NLP-based analysis and heuristic rules. While effective, it relies on a predefined sentence structure, limiting its ability to handle complex statements.

To reduce ambiguity in software requirements, Antoniou and Bassiliades (2024) propose Boilerplates — syntactic patterns linked to domain ontologies — to enforce semantic constraints. This structured approach supports clearer specifications and aids less experienced engineers through an integrated tool. Visual Narrator (Lucassen et al., 2017) generates conceptual models as Prolog or Web Ontology Language (OWL) but only captures entities and relationships, omitting class diagrams and attribute extraction. In contrast, Javed and Lin (2018) proposed a model from user stories and use cases with rules for attribute extraction, but these depend on a fixed verb set, limiting coverage.

UML Diagram Extraction through Requirements Analysis (UDRA) improves class modeling by integrating noun analysis, syntax, heuristics, and domain ontology, but its reliance on heuristics introduces bias and limits relationship coverage (Amune & Naik, 2016). The READ system (Bashir et al., 2021) extracts UML elements from English requirements using NLP and domain ontologies, yet it struggles with language ambiguities, affecting diagram completeness. Similarly, DC Builder automates UML model construction using NLP and heuristic rules but inherits their limitations (Herchi & Abdesslem, 2012). Overall, classical NLP, ontology-based, and heuristic methods offer useful insights but rely heavily on handcrafted rules and domain-specific knowledge, limiting their adaptability and ability to capture implicit meaning in natural language—particularly when handling complex, ambiguous, or informal requirements. In contrast, NMT offers a data-driven approach that learns contextual patterns from parallel data, allowing it to infer implicit information and resolve ambiguities. Framing UML generation as a translation task helps bridge the semantic gap more effectively than heuristic methods.

Recent advances in LLMs have enabled promising capabilities for transforming natural language requirements into UML class diagrams. LLMs demonstrate an understanding of domain concepts and can generate relevant modeling elements. However, a comparative study by Chen, Yang, et al. (2023) highlights ongoing challenges in producing complete and structurally accurate models. The study also shows that prompt design affects performance—examples can help, while reasoning steps may hinder results. These findings suggest that, despite their potential, LLMs alone are not yet sufficient for fully automated and reliable domain modeling. An experience report by Cámara et al. (2023) evaluating ChatGPT's performance in UML modeling reveals that its capabilities are still evolving and generally less robust than in class generation. The model struggles with certain advanced modeling concepts, such as association classes, and exhibits variability and inconsistency in generated outputs, which affects reproducibility. Additionally, the quality of generated models varies significantly depending on the problem domain. A study on UML sequence diagram generation finds ChatGPT promising but not yet a substitute for human engineers (Ferrari et al., 2024), highlighting the need for better handling of

complex requirements and suggesting Retrieval Augmented Generation (RAG) as a potential aid.

Chen, Yi, and Varró (2023) found that prompting works well for early-stage modeling, while fine-tuning is better for building constrained taxonomies, suggesting a hybrid approach combining both. Complementing this, a study by Chaaben et al. (2023) highlights few-shot learning as a valuable technique in data-scarce settings, showing that LLMs can adapt to new tasks with limited examples when guided by well-structured prompts. However, its effectiveness depends on the quality of examples and may decline with diverse or ambiguous inputs. While LLMs offer immense potential for automating and assisting in software modeling, they face three fundamental challenges — hallucinations, non-determinism, and bias — that require careful consideration. Addressing these issues calls for robust benchmarking frameworks and possibly novel techniques for LLM development and deployment in this critical domain (Cámara et al., 2024).

Despite prior efforts, a key gap remains in reliably translating unstructured and ambiguous requirements into accurate UML diagrams. Heuristic methods lack flexibility and struggle with complex or informal input (Herchi & Abdesslem, 2012, 2012; More & Phalnikar, 2012), while LLMs, though expressive, suffer from hallucinations and inconsistency (Alaswad et al., 2026a; Cámara et al., 2024, 2023). To address these limitations, our approach combines transformer-based models with structured representations to enhance consistency and reduce ambiguity in UML generation. Specifically, we leverage a transformer-based NLP pipeline that maps natural language software requirements to UML class diagrams. To bridge the gap between unstructured text and structured UML models, we adopt an intermediary language characterized by specific features designed to align the expressiveness of natural language with the formalism of class diagrams, including:

- Representing the diagram as a set of parts suitable for numerical embedding.
- Mapping a single natural language sentence to one or more statements.
- Being context-independent, allowing statements to be understood in any order.
- Including diverse predefined structures to avoid over- or under-representation of knowledge.

Considering these aspects, PlantUML was chosen for its ability to represent diverse relationships and entities. It is an open-source, domain-specific language (DSL) that uses clear, human-readable text to generate UML diagrams (Roques, 2015). Motivated by the limitations identified in existing literature, this study formulates the following research questions to guide a comprehensive evaluation of the proposed approach:

- RQ1:** How effectively can transformer-based models translate natural language software requirements into UML class diagram code in PlantUML syntax, while supporting sustainable and reliable software design practices?
- RQ2:** To what extent can transformer-based models enhanced with abstract syntax tree (AST) accurately translate natural language software requirements into UML class diagrams?
- RQ3:** How does the inclusion of frequent design patterns or structural priors influence the accuracy of translating natural language software requirements into UML class diagram elements?
- RQ4:** Can a fine-tuned transformer model achieve more stable, reproducible, and computationally efficient performance than zero-shot LLMs in generating UML class diagrams from natural language specifications?
- RQ5:** To what extent can the proposed structured translation framework generalize to other UML diagram types (e.g., sequence or use-case diagrams) while supporting scalable and sustainable model-driven engineering workflows?

3. Proposed methodology

We approach the automated generation of UML class diagrams from natural language requirements by framing it as an NMT task—a novel perspective that leverages advances in NLP. We explore two modeling strategies: a Sequence-to-Sequence (Seq2Seq) approach and a Sequence-to-Abstract-Syntax-Tree (Seq2AST) approach. Both approaches employ Transformer-based architectures but differ in their input-output representations and the degree of structural constraints they impose. Fig. 1 illustrates the main stages involved in transforming natural language into UML class diagrams.

3.1. Sequence-to-sequence model (Seq2Seq)

The Seq2Seq model is a well-established technique commonly utilized in various applications such as machine translation (Sutskever et al., 2014), text summarization (Rush et al., 2015), and dialogue systems (Vinyals & Le, 2015). The Seq2Seq approach considers the input as a natural language system description and generates the corresponding class diagram in PlantUML syntax as textual output. We use an encoder-decoder architecture based on Transformer. The encoder processes the input sentence and maps it to a latent semantic representation. The decoder then generates the PlantUML code token-by-token, conditioned on the encoder output and previous tokens.

The Transformer used in this work is based on T5. Specifically, we employ the CodeT5-small model, which has approximately 60 million parameters and offers a balance between efficiency and performance. It follows a standard encoder-decoder architecture with six layers, 512-dimensional hidden states, and 2048-unit feed-forward networks. The model uses learned relative positional embeddings, pre-layer normalization, and dropout for stability. The decoder applies causal masking and cross-attention. It is fine-tuned using cross-entropy loss and the Adam optimizer.

3.2. Sequence-to-AST model (Seq2AST)

3.2.1. Abstract syntax tree modeling for PlantUML code

Although PlantUML offers a structured modeling language, its textual format is highly variable and lacks a standardized representation, making it difficult for the models to consistently learn syntactic and semantic patterns. To address this, we introduce an intermediate representation in the form of an Abstract Syntax Tree (AST), structured according to a predefined schema (see Schema 1). This AST captures the hierarchical structure and semantics of the diagrams in a standardized format, enabling more consistent alignment between natural language and formal models, and facilitating a model-driven understanding of software requirements. The Seq2AST model is designed to translate natural language input into structured representations that can be incrementally composed into an AST, preserving both syntactic hierarchy and semantic meaning, as illustrated in Fig. 2.

Each UML class diagram code corresponding to a software requirement is transformed into a tree structure where the class entity serves as a parent node, branching into child nodes representing attributes, methods, and relations. This approach ensures a clear decomposition of diagram components, reflecting the structure found in software design. The schema specifies node types with required fields such as value, visibility, and data type, allowing precise modeling of elements and relationships. For instance, relation nodes not only capture connections between classes but also incorporate multiplicities and semantic labels, preserving key modeling nuances. Fig. 3 illustrates the process of transforming PlantUML code into an AST. It presents the corresponding software requirement snippet, its UML class diagram segment, the equivalent PlantUML code, and the resulting AST structure.

The use of AST brings several important advantages. First, it standardizes the output, ensuring all UML code adheres to a consistent and syntactically valid structure, regardless of input variations. Second,

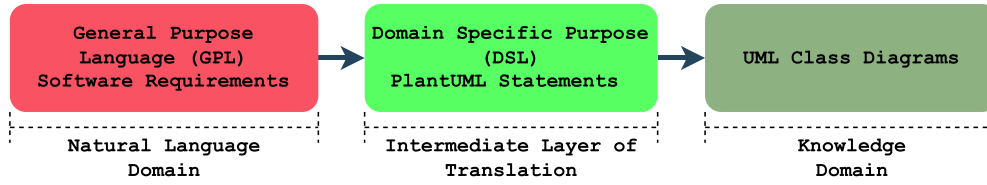


Fig. 1. Stages for the transition from natural language to the knowledge representation domain.

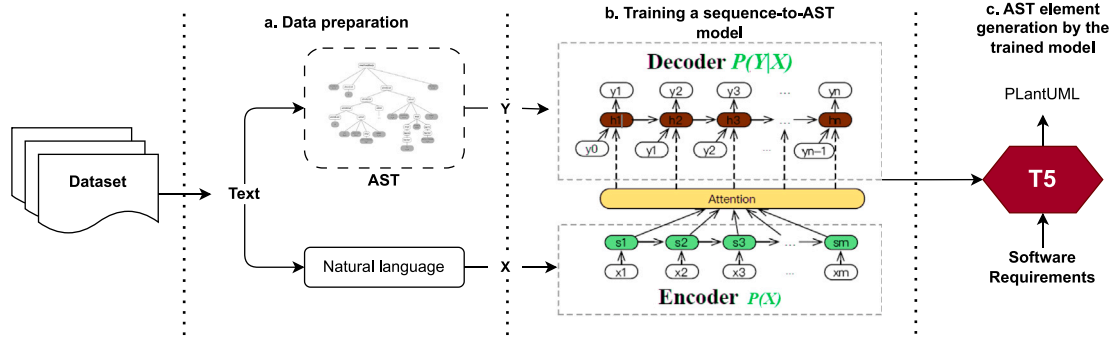


Fig. 2. Sequence to AST model.

Schema. 1 AST UML Schema Structure

Root Node:
 type = "root"
 children: List of {ClassNode}

ClassNode:
 type = "class"
 value: String
 children: List of {AttributeNode, MethodNode, RelationNode}

AttributeNode:
 type = "attribute"
 value: String
 visibility: String (default public)
 data_type: String (default string)

MethodNode:
 type = "method"
 value: String
 visibility: String (default public)
 data_type: String (default void)

RelationNode:
 type = "relation"
 value: String
 label: String
 multiplicity1: String
 multiplicity2: String
 children: Single ClassNode Reference
 Dependency: multiplicities (both must be defined together)

the tree structure allows the diagram to be traversed systematically from different perspectives—whether focusing on specific classes, relationships, or the overall architecture. Third, this representation improves the interpretability and modularity of the diagram, enabling granular analysis or targeted generation (e.g., regenerating only methods or relations). Finally, ASTs facilitate downstream processing and evaluation.

To ensure syntactic clarity and modeling consistency during AST construction, all class names follow a PascalCase format, while attribute and method names adopt camelCase. Primitive types are written in lowercase (e.g., string, int), whereas object types begin with capital

letters (e.g., Product, List). This standardization reduces token variability and simplifies tokenization, improving generalization during training. While camelCase may compact multiple words into a single token (e.g., startDate), subword tokenization mechanisms like SentencePiece or BPE split these into interpretable units (e.g., _start and Date), preserving semantic associations. This balance between compactness and decomposability supports both syntactic usability and semantic understanding, acting as a form of syntax-level supervision that improves training stability and reduces outlier patterns.

3.2.2. Multi-task learning with grammar-constrained AST-based output

After constructing the AST for each individual requirement, the resulting structure can be leveraged from a task-oriented perspective. Instead of treating the AST as a single-output format, it can be repurposed across multiple learning objectives by presenting the same requirement and its corresponding AST within different task settings. This multi-task framing enables the model to learn a shared underlying representation of the software requirement while specializing in distinct sub-tasks. As a result, the model not only generalizes better across diverse software modeling scenarios but also develops a deeper understanding of the syntax and semantics embedded in the structured output. The full training pipeline — including the AST parser, schema definitions, dataset preprocessing, and training configuration — is available at <https://github.com/FeisalAlaswad/GenClassSeq2AST>.

Fig. 4 illustrates how a single requirement is decomposed into multiple task-specific training instances. In addition to the original requirement, each task receives a shared contextual representation derived from the specification, consisting of extracted entities and actions. For the requirement shown in Fig. 4, the extracted shared context includes:

Entities: customer, order, product, payment, delivery, address, history, inventory, administrator, category, shipment.

Actions: browse, place, track, register, record, contain, select, store, include, confirm, validate, calculate, assign, deliver, provide, make, associate, view, update, manage, process, monitor, maintain.

In the class extraction task shown at the top of Fig. 4, the model receives only the shared context and requirement, without a target class

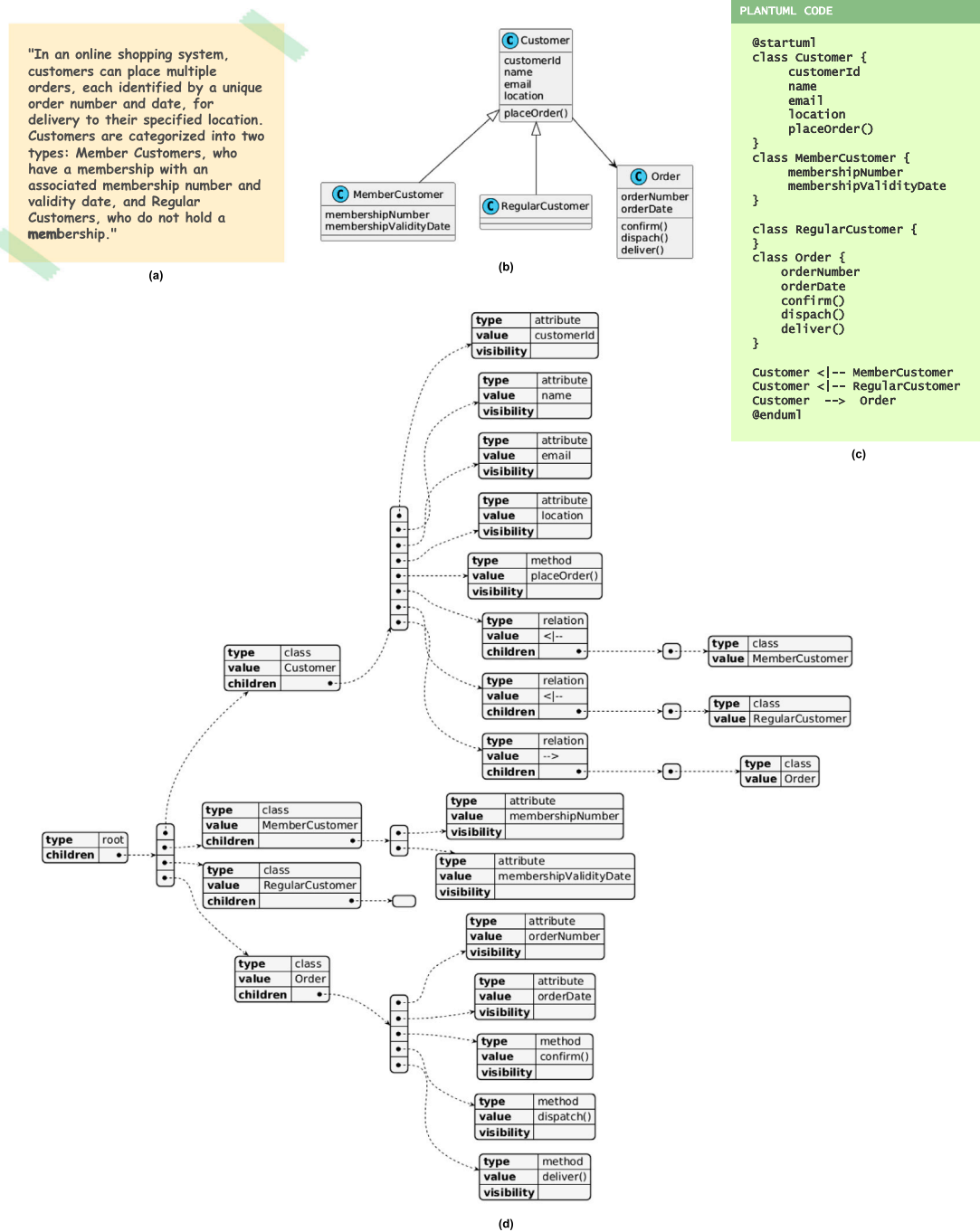


Fig. 3. Transforming the PlantUML code into an AST: (a) Snippets of the software requirements; (b) Corresponding class diagram segment; (c) Equivalent PlantUML code; (d) AST representation of the PlantUML code.

specification, and outputs the identified class names. In the method extraction task shown in the middle of Fig. 4, a [CLASS] token is added to the input to anchor the prediction to a specific class—in this example, Customer. The model then outputs the corresponding methods, including their visibility and return types, in AST format. In the attribute extraction task shown at the bottom of Fig. 4, the same anchoring strategy is applied to a different class (Order), and the model outputs its typed attributes. This input design enables each task to be learned independently while allowing the shared context block to provide global domain information across all tasks.

Each output task adheres to a strict grammar structure to ensure consistency and ease of parsing. The output grammar is rigorously defined to enforce conformance, covering rules for classes, attributes,

methods, relations, and visibility modifiers, as illustrated in Fig. 5. This grammar not only guides the structure of the output but also standardizes naming conventions — such as using PascalCase for class names and camelCase for attributes and methods — to reduce variability and improve model generalization.

Fig. 5 presents the complete grammar tree that defines the structured format expected in the Transformer model's target output. Throughout both training and inference, the model strictly adheres to this grammar to produce syntactically valid and semantically consistent structured representations. Reading from left to right, the grammar first specifies structural nodes (classes), followed by property nodes (attributes and methods), and finally relational nodes. Within each category, the grammar prescribes both the token sequence and naming

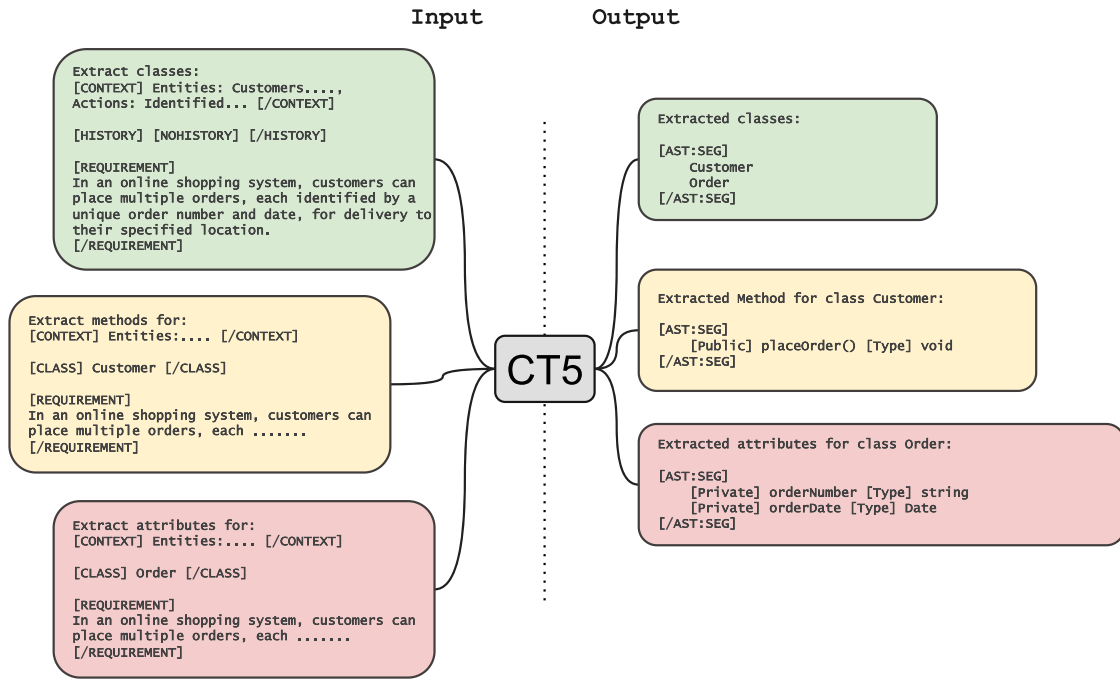


Fig. 4. Multi-task decomposition of a single software requirement into task-specific training samples. The left column shows three distinct input prompts, each targeting a different identification task (class, method, and attribute extraction). Each prompt combines a task-specific prefix, a global context block summarizing the domain entities and actions extracted by KeyBERT, an optional history block tracking previously identified classes, and the natural language requirement itself. The right column shows the corresponding structured AST outputs produced by the Seq2AST model for each task.

conventions. For example, a method node always follows the pattern [METHOD] visibility method_name [TYPE] return_type, where method_name is represented as a camelCase string and may optionally include parameter information (e.g., placeOrder()). Similarly, relational nodes require multiplicity tokens to appear as a pair (e.g., [ONE] [MANY]), ensuring that relationship cardinalities are specified consistently. By enforcing these structural constraints, the grammar guarantees that the generated output can be directly parsed into an Abstract Syntax Tree (AST), facilitating seamless reconstruction into UML class diagram languages such as PlantUML. This grammar-constrained generation strategy is a key mechanism by which Seq2AST avoids the syntactic inconsistencies and malformed outputs commonly observed in unconstrained sequence-to-sequence generation approaches.

The grammar presented in Fig. 5 provides a high-level abstracted representation intended to illustrate the core structural syntax used during training and inference. The proposed grammar primarily focuses on the most frequently occurring UML class diagram constructs, including association, inheritance/extension, aggregation, composition, and dependency relations. While PlantUML supports additional specialized variants and notational extensions, several of these occur infrequently in benchmark datasets and may introduce structural sparsity and class imbalance during learning. Furthermore, semantically related constructs are internally normalized to unified representations (e.g., inheritance and extension). The implementation-level parser additionally models relation directionality and semantic variants through dedicated structured tokens, although these low-level mappings are omitted from Fig. 5 for readability and compactness.

The model architecture is built upon fine-tuning a CodeT5 model for four dedicated tasks: Class Identification, Attribute Identification, Method Identification, and Relation Identification. Each input to the model consists of a task-specific prefix, contextual information, the natural language requirement, and — for tasks other than Class Identification — the target class. To maintain semantic continuity across extractions, a history component tracks the classes that have been previously identified. The context consists of a set of derived

entities and verbs extracted from the requirement texts within each software scenario, using the pretrained KeyBERT transformer. During this process, the most frequent and less informative tokens in each scenario (e.g., system, attribute, etc.) are removed to better capture the specific characteristics of the scenario's requirements. The resulting context is not intended as a feature-engineering representation but rather as a compact semantic summary of the software domain. Since the proposed framework processes each requirement independently, relevant domain information may be distributed across multiple requirements and therefore unavailable to the model when a single requirement is considered in isolation. To mitigate this limitation, the extracted context is supplied alongside each requirement to provide a global view of the system under development. We employ KeyBERT rather than frequency-based techniques such as TF-IDF because it identifies semantically representative concepts using contextual transformer embeddings, allowing the generated context to better capture domain meaning and conceptual relationships beyond simple term frequency statistics.

3.2.3. Enriching UML generation using frequent design structures

Even in the absence of large annotated datasets for directly mapping natural language (NL) to UML models, a promising alternative emerges. An extensive corpus of over 172,682 structured class diagrams offers a rich source of structural design knowledge. These models, drawn from diverse domains, encapsulate expert decisions and reflect widely adopted modeling conventions. While supervised learning alone may struggle to capture the diversity and complexity of real-world software designs, we argue that this wealth of unlabeled design data can be strategically leveraged to enrich data-driven pipelines and reduce reliance on paired textual supervision.

To exploit this data, we first parse each class diagram into a structured AST, capturing its constituent components—namely, classes, attributes, methods, and inter-class relationships. These elements are then transformed into transactional datasets, where each transaction represents a context such as the attributes of a class, the methods of

```

[AST:SEG]
|-- content
| |-- class*
| | +-- [CLASS] class_name
| | +-- class_name -> PascalCase
| | -- attribute*
| | +-- [ATTRIBUTE] visibility attribute_name type_declaration
| | | -- visibility
| | | +-- [PUBLIC] | [PRIVATE] | [PROTECTED] | [PACKAGE]
| | | -- attribute_name -> camelCase
| | +-- type_declaration
| | +-- [TYPE] type_name
| | +-- type_name -> collection_type | primitive_type | PascalCase
| -- method*
| +-- [METHOD] visibility method_name type_declaration
| | -- visibility (same as above)
| | -- method_name -> camelCaseWithParameters
| | -- type_declaration (same as above)
| -- relation*
| +-- (association | extension | aggregation | composition | dependency)
| -- relationship
| +-- [RELATION] class_name multiplicity multiplicity class_name label
| | -- class_name -> PascalCase
| | -- multiplicity -> [ONE] | [MANY] | [ZERO] | [MOM] | [OOM]
| | -- multiplicity (again)
| | -- class_name (again)
| | +-- label
| | +-- [LABEL] label_text
| | +-- label_text -> camelCase | [NOLABEL]
| -- extension
| +-- [EXTENSION] class_name [ONE] [ONE] class_name label
| (same internal structure as relationship)
| -- aggregation
| +-- [AGGREGATION] class_name multiplicity multiplicity class_name label
| (same internal structure)
| -- composition
| +-- [COMPOSITION] class_name multiplicity multiplicity class_name label
| (same internal structure)
| -- dependency
| +-- [DEPENDENCY] class_name multiplicity multiplicity class_name label
| (same internal structure)
[/AST:SEG]

```

Fig. 5. Grammar tree defining the structured output format used for training and inference. The tree is rooted at an [AST:SEG] block and organizes the output into class, attribute, method, and relation nodes with fixed syntactic constraints, enabling deterministic parsing into an AST.

a class, or the set of relationships associated with a class within a diagram.

To operationalize the insights mined from the large corpus of class diagrams, we train the model on three auxiliary tasks designed to enhance the structural completeness of UML class diagrams: Attribute Suggestion, Method Suggestion, and Relation Suggestion. These tasks are formulated to predict likely but missing elements based on context, drawing on recurring co-occurrence patterns observed in the source data. Rather than relying on explicit natural language requirements, the model learns to infer structurally plausible attributes, method groupings, and inter-class relationships that frequently appear in similar diagram contexts.

For example, Fig. 6 illustrates the attribute suggestion task using a banking-domain scenario. Given only the target class *Bank* and a set of contextual class names, the model generates a set of typed attributes, including *name*, *code*, *headquarters*, *swiftCode*, and *licenseNumber*. These suggestions are derived from learned structural patterns within the corpus rather than from textual cues, enabling the model to act as a domain-aware completion mechanism. Furthermore, the model predicts appropriate visibility modifiers for the generated attributes, reflecting common design conventions learned from existing UML diagrams. This capability complements the identification tasks by enriching generated class diagrams with structurally meaningful elements that may be omitted, underspecified, or only implicitly described in the original requirements.

These auxiliary tasks are integrated during training to complement the primary NL-to-UML generation task, reinforcing domain-aware generation. This strategy leads to UML diagrams that are not only syntactically valid but also more semantically complete and aligned with expert design intuition.

Given the imbalance between the suggestion and identification task datasets, we adopt a two-stage fine-tuning strategy. The model is first trained on the suggestion tasks to capture underlying structural design patterns, followed by fine-tuning on the identification tasks to map natural language inputs to explicit diagram components. This sequential approach enables the model to first internalize domain knowledge

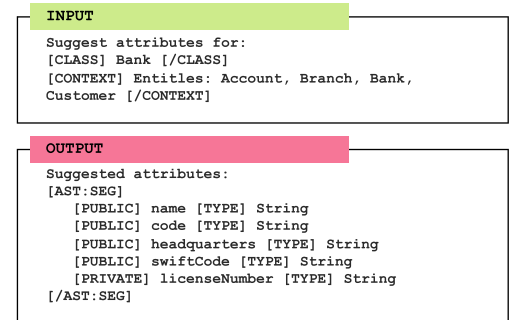


Fig. 6. Example of the attribute suggestion task for the class *Bank*. The input prompt (top) provides the target class and its contextual classes through [CLASS] and [CONTEXT] tokens. Without any natural language requirement, the model relies on structural patterns learned from the ModelSet corpus to generate typed attributes in AST format (bottom). The suggested attributes, such as *swiftCode* and *licenseNumber*, reflect common banking-domain concepts inferred from co-occurrence patterns, demonstrating how the suggestion module enriches class diagrams with semantically plausible elements beyond those explicitly specified in requirements.

before tackling language understanding. To mitigate the risk of catastrophic forgetting — common in multi-phase fine-tuning — we evaluate the model's performance on the suggestion tasks after the second stage. The model maintains its earlier performance, demonstrating strong generalization across both structural and linguistic tasks.

3.2.4. Inference and AST reconstruction

This stage presents a structured methodology for translating unstructured software requirements into formal design representations through a sequence of machine learning-driven inference steps and syntax-guided reconstruction. Algorithm 1 outlines an automated pipeline for constructing ASTs representing UML class diagrams from natural language software requirements. It begins with a preprocessing phase, where each requirement is cleaned by removing non-informative

Algorithm. 1 AST Construction from Natural Language Requirements

```

1: Input: Natural language software requirements
2: Output: AST representing UML class diagram
3:
4: Preprocessing Phase:
5: for each requirement  $r$  in requirements do
6:    $r_p \leftarrow \text{removeStopNonInformativeTokens}(r)$ 
7:   Add  $r_p$  to  $R_p$ 
8: end for
9:
10:  $\text{context} \leftarrow \text{extractGlobalContext}(R_p)$ 
11:
12: Elements Identification and Suggestion:
13:  $\text{AST} \leftarrow \text{initializeEmptyAST}()$ 
14: Initialize empty class set  $C$ 
15: for each requirement  $r_p$  in  $R_p$  do
16:    $\text{classes} \leftarrow \text{classesIdentification}(\text{context}, C, r_p)$ 
17:   Add  $\text{classes}$  to  $C$  (if not already exist)
18:   for each class  $c$  in  $\text{classes}$  do
19:     if  $c$  not in  $\text{AST}$  then
20:       addNodeToAST( $\text{AST}$ ,  $c$ )
21:     end if
22:      $\text{attrs} \leftarrow \text{attributesIdentification}(\text{context}, c, r_p)$ 
23:      $\text{methods} \leftarrow \text{methodsIdentification}(\text{context}, c, r_p)$ 
24:      $\text{relations} \leftarrow \text{relationsIdentification}(\text{context}, c, r_p)$ 
25:     addNodesToClass( $c$ ,  $\text{attrs}$ ,  $\text{methods}$ ,  $\text{relations}$ )
26:   end for
27: end for
28:
29: for each class  $c$  in  $C$  do
30:    $\text{attrs} \leftarrow \text{attributesSuggestion}(\text{alphabetize}(C), c)$ 
31:    $\text{methods} \leftarrow \text{methodsSuggestion}(\text{alphabetize}(C), c)$ 
32:    $\text{relations} \leftarrow \text{relationsSuggestion}(\text{alphabetize}(C), c)$ 
33:   addNodesToClass( $c$ ,  $\text{attrs}$ ,  $\text{methods}$ ,  $\text{relations}$ )
34: end for
35:
36: return  $\text{AST}$ 

```

tokens. A global context is then extracted using KeyBERT to support consistent inference. The pipeline initializes an empty AST and iteratively processes each requirement to identify relevant classes. For each class, it identifies attributes, methods, and inter-class relations, adding them to the AST. Suggestion modules refine these elements based on the evolving class set. The final AST provides a structured, merged view of all design elements, enabling seamless translation into PlantUML.

Algorithm 1 incrementally constructs a single global AST from all requirements belonging to the same software project. Rather than generating independent class diagrams for individual requirements, the framework processes requirements sequentially and continuously updates the shared AST. The class set C and the evolving AST preserve information extracted from previously processed requirements, while the history component supplied to the model provides contextual awareness of already discovered classes. This history mechanism is used exclusively during inference to guide the model toward consistent class reuse and to reduce duplicate class generation. Once inference is complete, the conversion from AST to PlantUML is fully deterministic. When a new requirement introduces a class that already exists in the AST, its attributes, methods, and relations are merged into the corresponding class node, while duplicate elements are removed through schema validation. For example, if Requirement 1 yields a `Customer` class with attribute `name`, and Requirement 5 yields the same `Customer` class with attribute `email` and a relation to `Order`, the final AST node for `Customer` will contain both `name`, `email`, and the relation—accumulated from both requirements into a single integrated class. Because PlantUML is an order-independent declarative language, the final class diagram is unaffected by the order in which requirements are processed or UML elements are discovered.

Consequently, information extracted from different requirements can be safely accumulated into a single integrated model.

In cases where multiple requirements yield predictions for the same UML element, the framework follows the incremental AST update process defined in Algorithm 1. PlantUML tolerates multiple declarations when they are structurally distinct. For example, two attributes sharing the same name but differing in type are treated as separate declarations rather than being automatically merged. Consequently, both elements are retained in the generated diagram, and the conflict appears as a redundant but syntactically valid modeling construct. For instance, if one requirement leads to the prediction `phoneNumber : Integer` while another yields `phoneNumber : String` for the same class, both attributes will be preserved in the final PlantUML representation. Such cases are retained as redundant but syntactically valid elements in the generated model and are explicitly accounted for during evaluation. In contrast, when multiple predictions correspond to the same declaration (i.e., they are not structurally distinct), the later declaration effectively supersedes the earlier one during AST reconstruction. We further clarify that the framework does not rely on explicit terminology normalization rules. Instead, the history context reduces the likelihood of terminological inconsistency by exposing previously discovered class names to the model at inference time, encouraging reuse of existing identifiers rather than generating new variants. We acknowledge that this does not constitute a guaranteed normalization procedure; cases where divergent identifiers are generated for the same concept are treated as redundant but syntactically valid declarations and are explicitly accounted for during evaluation. This incremental merging strategy enables the framework to integrate information distributed across multiple requirements while maintaining a coherent UML class diagram for the entire software project.

4. Experimental setup and results

4.1. Datasets preparation and augmentation

Our model addresses two primary tasks: (1) identifying class diagram elements from NL requirements, and (2) suggesting class diagram elements based on contextual understanding. To achieve these objectives, we employed two separate datasets tailored to each task.

4.1.1. Natural language to PlantUML dataset

The PlantUCD (PlantUML Class Diagram) dataset (Alaswad et al., 2026b) was used for class diagram element identification. This curated collection was compiled from diverse software requirements sourced from Stack Overflow, TutorialsPoint, and academic literature. In addition, an annotated subset of class diagrams from the AtlanMod Zoo (Atlanmod, 2020) was incorporated using the annotations provided by Songyang-dev (2022).

This dataset was collected through a supervised university project involving 12 contributors. Participants from various academic levels submitted class diagrams along with corresponding natural language requirements and PlantUML code via a structured Google Form. Over a two-month period, 618 submissions were gathered. A quality-control process was subsequently applied to ensure dataset consistency. Submissions were excluded if they contained incomplete requirements (i.e., fewer than two sentences or descriptions involving fewer than two distinct classes), syntactically invalid PlantUML code (i.e., code that failed to parse or produced an empty diagram), duplicate entries, diagrams unrelated to the accompanying requirements (i.e., requirement–diagram pairs identified during the quality-control process as semantically misaligned, where the entities or relationships described in the requirements were not reflected in the submitted model), or models lacking essential UML elements such as attributes, methods, or relationships (i.e., diagrams consisting solely of class definitions without sufficient structural information for fine-grained analysis). Disagreements were resolved through discussion, and pairs for which consensus

Table 1
Overview of datasets used for class diagram element identification and suggestion in the proposed work.

Subset	UML models	Raw requirements	Augmented raw requirements	Task-level samples
Identification Task:				
Training	147	3777	8134	71 066
Testing	30	853	1409	12 274
Suggestion Task:				
Training	5960	–	–	104 751
Testing	2554	–	–	44 839

could not be reached were excluded. Following this filtering stage, 147 UML class diagrams and their associated requirements were retained for training, while 30 diagrams were reserved for testing.

Because UML modeling can admit multiple valid representations for the same requirement, some submissions exhibited variations in naming conventions, class decomposition, and relationship modeling. Rather than enforcing a single modeling style, the dataset preserves contributor-provided representations, provided they satisfied the quality-control criteria and remained semantically consistent with the accompanying requirements. For example, the requirement “A student can enroll in multiple courses” may be represented either as a direct many-to-many association between Student and Course or through an intermediate Enrollment class. Such variations naturally occurred across different submissions and were retained provided that the resulting UML model accurately captured the semantics expressed in the accompanying requirement. All models are formatted in the PlantUML language and are publicly available on GitHub (Alaswad, 2025). The requirements fall into two types: Descriptive Requirements (implicit), which describe system behavior and imply class diagram elements without explicitly mentioning them, and Declarative Requirements (explicit), which directly specify these elements. To meet the transformer’s data needs, data augmentation was performed by rephrasing the natural language requirements. When rephrasing affected the structure or terminology of the corresponding PlantUML code, necessary adjustments were made to maintain alignment (see Appendix for examples).

Our dataset after augmentation consisted of 8134 raw training samples. Since we adopt a multitask learning approach, each raw sample is transformed into multiple training instances based on the specific tasks defined. In our setting, each sample can yield several distinct training examples corresponding to different task formulations, such as class, attribute, and method identification. This expansion results in a final training set of approximately 71,066 samples. An overview of the dataset composition, including the number of UML models, raw and augmented requirements, and the resulting task-level training samples, is presented in Table 1.

4.1.2. Class diagram element suggestion dataset

The ModelSet dataset (López et al., 2022) is a large-scale collection of over 172,682 structured class diagrams, automatically derived from approximately 5432 Ecore models and 3085 XMI files,¹ and further processed into a curated, labeled corpus,² serving as a valuable resource for structural design insights. These models originate from a wide range of domains and demonstrate diverse modeling conventions. However, not all files are usable—some are in non-English languages, and others lack detailed information, containing only class definitions without attributes or relationships, which restricts their applicability for fine-grained element analysis (see Table 1 for details).

¹ Subset selected from the full 5460 Ecore and 5120 UML models available.

² <https://github.com/FeisalAlaswad/ModelSet-AST-Structured-JSON>

4.2. Evaluation methods and metrics

Our model employs the BLEU-4 (Bilingual Evaluation Understudy) score, commonly used in machine translation, to assess the quality of generated statements (Papineni et al., 2002).

Evaluation criteria discussed in Almazroi et al. (2022) are employed in assessing the proposed method for creating a class diagram. The Recall percentage refers to the system’s ability to generate all components of a class diagram, as in Eq. (1).

$$Recall = \frac{N_{correct}}{N_{correct} + N_{missed}} \quad (1)$$

Where $N_{correct}$ stands for the count of accurately identified components, N_{miss} represents the number of components identified by the human expert (ground truth) but overlooked by the proposed approach. The precision percentage refers to the accuracy of the components recognized in the proposed approach can be calculated as in Eq. (2).

$$Precision = \frac{N_{correct}}{N_{correct} + N_{incorrect}} \quad (2)$$

$N_{incorrect}$ means the correct components are classified as wrong. Finally, the F1 score is computed as shown in Eq. (3).

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (3)$$

4.3. Experimental design and evaluation results

To systematically assess the effectiveness and limitations of the proposed approach, this section reports and analyzes results with respect to the research questions introduced in Section 2.

4.3.1. Addressing RQ1 and RQ2 (Seq2Seq vs. Seq2AST)

We compared a traditional Seq2Seq model with our proposed Seq2AST model. The BLEU scores reveal a clear performance gap: Seq2Seq achieved a score of 38.67, while Seq2AST scored 56.38 on the Identification task, demonstrating the feasibility of translating natural language requirements into PlantUML. On the Suggestion task, Seq2AST achieved a BLEU score of 53.12. However, since BLEU primarily measures token overlap, it does not adequately capture structural or semantic correctness—limiting its ability to reflect UML fidelity.

For a more rigorous structural evaluation, we employed Recall, Precision, and F1-Score (Section 4.2). The Seq2AST model achieved high scores in all three metrics: Recall (88.62), Precision (82.13), and F1-Score (85.25), outperforming the Seq2Seq model, which recorded Recall (74.16), Precision (68.21), and F1-Score (71.01). These results demonstrate that using ASTs significantly improves the structural quality of the generated diagrams. Nevertheless, these quantitative metrics do not distinguish between critical and minor modeling errors, highlighting the need for further qualitative evaluation.

Table 2
Performance comparison of the Seq2AST model across different UML component enhancements.

Approach	Enhancement type	Recall (%)	Precision (%)	F1-score (%)
Seq2AST	None	88.62	82.13	85.25
	Attributes	90.51	85.37	87.86
	Methods	90.20	83.80	86.88
	Relations	88.84	82.72	85.67
	Integrated	92.31	87.64	89.91

4.3.2. Addressing RQ3: impact of structural enhancements

Integrating structural priors from Section 3.2.3 into Seq2AST significantly improved its performance. The best configuration — a fully integrated model — achieved high scores in Recall (92.31%), Precision (87.64%), and F1-Score (89.91%). Table 2 illustrates the impact of applying enhancements to individual model components. Importantly, while each enhancement targets a specific aspect of the model, evaluation is conducted on the entire generated diagram. These results confirm that incorporating design patterns and structural priors improves the overall completeness and accuracy of generated diagrams. Integrating structural enhancements into Seq2AST led to notable gains, improving Recall (3.69%), Precision (5.51%), and F1-Score (4.67%) over the baseline. This confirms the benefit of incorporating design patterns and structural priors for generating accurate and complete UML diagrams.

Due to its relatively low accuracy and the need for manual correction of PlantUML code, the Seq2Seq model was excluded from further evaluation. Evaluating domain models is inherently subjective, as a single requirement can yield multiple valid representations. Different UML designers may produce equally valid class diagrams from the same requirement specification. Variations may arise in class decomposition, abstraction level, relationship modeling, or the introduction of intermediary classes, while still preserving the intended semantics of the domain. This subjectivity highlights the need for structured, semantics-aware evaluation metrics that assess diagram correctness and completeness beyond strict syntactic similarity. Naming conventions also vary across individuals and projects, further motivating the need for evaluation metrics that prioritize semantic correctness and structural completeness. Such metrics should account for the presence of extra elements — whether harmless or harmful — rather than relying solely on syntactic similarity or assuming the mere existence of relevant elements. To mitigate evaluation bias, we employed a blind testing protocol in which each contributor assessed randomly assigned diagrams without knowledge of the generating model. The evaluation was conducted by nine assessors drawn from SRM Institute of Science and Technology, Brescia University, and University of Aleppo. All participants had at least three years of background in software engineering with specific experience in UML modeling. Four assessors held postgraduate degrees in software engineering or information systems analysis, and five held doctoral degrees with extensive academic and practical experience in UML modeling. All assessors had previously used UML tools, including PlantUML. Diagrams were assigned randomly under a blind protocol, ensuring that no assessor had prior knowledge of the generating model. The evaluation set comprised 36 software requirements—30 with existing ground-truth diagrams and 6 without publicly available or well-defined references. These six requirements were chosen to reflect real-world specifications, containing informal language, ambiguity, domain-specific terminology, and implicit design decisions. For the latter, contributors independently created reference diagrams based on their domain understanding to support a fair and meaningful evaluation. Each generated diagram was scored based on two key metrics:

Correctness Measures the accuracy of elements, rewarding valid additions and penalizing incorrect ones. As given in Eq. (4):

$$\text{Correctness} = \frac{C_{TP} + C_{Valid} - C_{Harmful} + D}{2D} \quad (4)$$

- C_{TP} : Correct elements that fully match the reference
- C_{Valid} : Extra but valid elements
- $C_{Harmless}$: Extra but harmless elements
- $C_{Harmful}$: Extra and harmful elements
- D : Total elements

$$D = C_{TP} + C_{Valid} + C_{Harmless} + C_{Harmful}$$

Completeness Measures how many essential elements from the reference are present in the generated model, as given in Eq. (5). Only elements that are fully correct — matching in both type and structure — are counted. Partial or approximate matches are excluded to ensure strict semantic alignment.

$$\text{Completeness} = \frac{E_{Full-Matched}}{E_{GT}} \quad (5)$$

- $E_{Full-Matched}$: Required elements correctly and fully generated
- E_{GT} : Total required elements in the reference diagram

Correctness and Completeness are mathematically defined and computed, but rely on human interpretation during annotation or assessment—making them quantitative metrics grounded in qualitative evaluation. Table 3 presents the evaluation of the proposed Seq2AST (Integrated) model across key aspects of class diagram generation. For each perspective, we report both completeness and correctness scores, which reflect how thoroughly the model captures the expected elements and how accurately those predictions match the ground truth, respectively (see the complete evaluation example³). The overall scores are reported as micro-averages computed over all evaluated diagram elements rather than as arithmetic averages of the individual component-level scores. Although these scores are computed numerically, they are designed to assess qualitative aspects of diagram quality, such as semantic alignment, structural integrity, and element validity.

4.3.3. Addressing RQ4

To evaluate the effectiveness of our fine-tuned transformer model, we compared Seq2AST against several state-of-the-art zero-shot LLMs using identical natural language inputs. LLM baselines were evaluated in a zero-shot setting to maintain a controlled and comparable evaluation protocol with the proposed Seq2AST framework. Advanced prompting strategies such as Chain-of-Thought or few-shot prompting were intentionally excluded, as the proposed T5-based architecture performs direct structured generation without iterative prompt-driven reasoning. Allowing iterative prompt correction or refinement would shift the evaluation from model-driven generation to prompt-driven generation, where performance becomes highly dependent on prompting strategies, user expertise, and UML design knowledge. Under such conditions, defining fair and reproducible correction rules becomes challenging, as increasingly detailed corrective prompts can progressively steer the model toward near-perfect outputs regardless of its intrinsic generation capability. Consequently, the comparison focuses on intrinsic generation capability under equivalent conditions rather than prompt-engineering optimization.

Unlike Table 2, where Recall, Precision, and F1-score are computed over all UML elements collectively (micro-average), Table 4 reports

³ <https://github.com/FeisalAlaswad/GenClass-Running-Example>

Table 3

Qualitative metrics for the proposed Seq2AST model across different aspects of the class diagram.

Perspective	Completeness (%)	Correctness (%)
Class	83.6	84.8
Attribute	89.9	86.9
Method	84.0	83.0
Dependency	69.3	71.1
Extension	72.2	70.1
Association	71.0	69.4
Aggregation	65.2	68.5
Composition	64.4	66.7
Overall (Micro-Avg.)	77.4	78.2

Table 4

Component-wise performance comparison of UML diagram generation across different LLMs models.

Model	Class			Attributes			Method			Relationship			Diagram (Macro Avg.)		
	Rec.	Prec.	F1	Rec.	Prec.	F1	Rec.	Prec.	F1	Rec.	Prec.	F1	Rec.	Prec.	F1
GPT-3.5-turbo	94.05	87.63	90.73	93.86	88.40	91.05	91.65	86.97	89.25	90.36	87.27	88.79	92.48	87.57	89.95
GPT-4-turbo	96.13	91.96	94.00	94.33	89.19	91.69	92.97	88.87	90.87	92.76	90.73	91.73	94.55	90.69	92.07
DeepSeek-V3	95.74	91.77	93.71	93.97	89.45	91.65	90.96	87.47	89.18	91.83	89.96	90.89	93.63	89.67	91.36
LLaMA-3	94.96	90.03	92.43	92.51	87.94	90.17	91.97	87.25	89.55	90.67	87.91	89.27	92.53	88.78	90.35
Gemini 1.5	93.55	88.57	90.99	91.97	87.49	89.67	91.37	85.85	88.52	90.17	87.63	88.88	91.27	87.39	89.52
Gemini 1.5 Flash	89.37	87.57	88.46	91.47	84.87	88.05	90.57	84.77	87.57	90.17	86.49	88.29	90.40	85.93	88.09
Seq2AST	94.16	88.27	91.12	91.97	86.37	89.08	90.77	85.47	88.04	90.07	87.06	88.54	91.74	86.79	89.20

Table 5

Comparison of Seq2AST and LLMs in terms of class diagram completeness and correctness.

Model	Completeness (%)	Correctness (%)
GPT-3.5-turbo	79.6	78.0
GPT-4-turbo	83.3	81.6
DeepSeek-V3	82.5	80.9
LLaMA-3	78.6	79.8
Gemini 1.5	77.2	78.2
Gemini 1.5 Flash	76.8	77.3
Seq2AST (Integrated)	77.4	78.2

component-wise results and computes the overall diagram score as the macro-average of the Class, Attribute, Method, and Relationship categories. Consequently, the overall Seq2AST values reported in the two tables may differ slightly.

As shown in Table 4, Seq2AST achieved a macro-averaged F1-score of 89.20%, outperforming Gemini 1.5 Flash (88.09%) and remaining competitive with models such as Gemini 1.5 (89.52%), GPT-3.5-turbo (89.95%), and LLaMA-3 (90.35%). Although GPT-4-turbo (92.07%) and DeepSeek-V3 (91.36%) achieved slightly higher scores, the performance gap remains modest, underscoring Seq2AST's ability to rival general-purpose LLMs in structured diagram generation.

Furthermore, the qualitative analysis in Table 5 underscores Seq2AST's capability to generate structurally valid UML class diagrams. While leading LLMs such as GPT-4-turbo and DeepSeek-V3 achieve higher correctness scores (81.6% and 80.9%, respectively), Seq2AST maintains competitive correctness (78.2%) and completeness (77.4%) scores. Its performance is comparable to LLaMA-3, Gemini 1.5, and Gemini 1.5 Flash, demonstrating that a fine-tuned, structure-aware model like Seq2AST can match or exceed several zero-shot LLMs in terms of semantic and structural fidelity.

4.4. Error analysis for component-level performance gap

The aggregate scores in Table 3 reveal a consistent and statistically meaningful performance gap between *structural* components (classes, attributes, methods) and *relational* components (dependency, extension, association, aggregation, composition). Structural components achieve completeness scores above 83%, whereas relational components fall below 73%, with aggregation (65.2%) and composition (64.4%) representing the lowest scores across all evaluated perspectives. This section presents a fine-grained error analysis that identifies the root causes of this disparity through a taxonomy of recurring failure modes, supported by representative examples.

4.4.1. Error taxonomy and classification

Errors in the generated class diagrams were manually categorized by reviewing a stratified sample of 180 mismatched elements drawn from the test set covering all five relational types. Each mismatch was assigned to one or more of the following four error categories, which were derived inductively from inspection of model outputs against ground-truth diagrams.

E1 — Structural sparsity (missing relations entirely): The model fails to generate the relation at all, typically because the corresponding natural language cue is expressed implicitly rather than with an explicit verb or keyword. For example, a requirement such as “Each order is composed of line items” reliably triggers relation generation, whereas “An order holds its items” often does not. Composition and aggregation are particularly susceptible because their natural language triggers overlap heavily with simple possession or containment verbs (*has*, *contains*, *holds*), which the model frequently resolves as simple attributes rather than structural relationships.

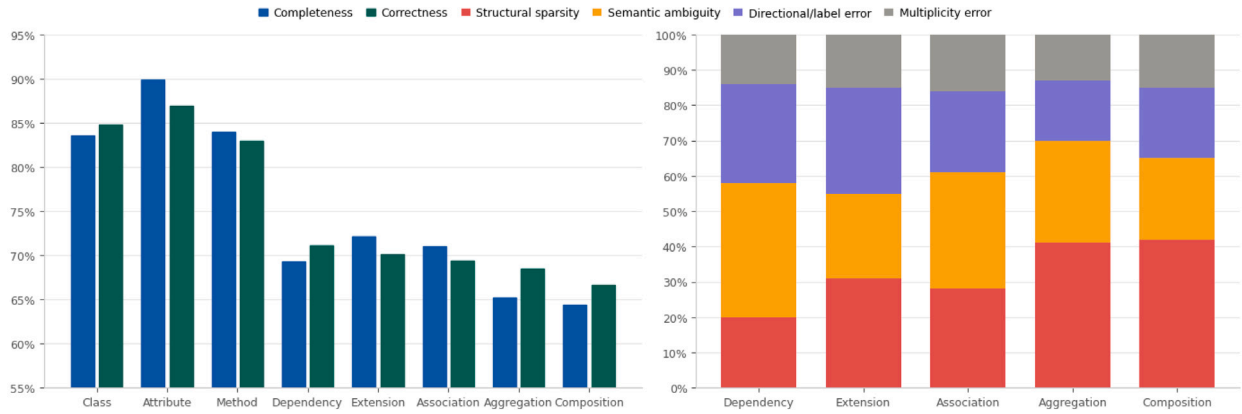
E2 — Semantic ambiguity (wrong relation type): The model generates a relation between the correct pair of classes but assigns the wrong type, most commonly conflating association with aggregation, or aggregation with composition. This conflation reflects a genuine ambiguity in natural language: the distinction between “A department has employees” (aggregation: employees exist independently) and “A house has rooms” (composition: rooms cannot exist without the house) is not recoverable from surface form alone without world knowledge or explicit domain cues. Dependency relations are especially vulnerable to this error because the phrase “depends on” is semantically broad and appears in both functional and structural contexts.

E3 — Directional or label errors: The model generates the correct relation type and class pair but reverses the directionality (e.g., generating

Table 6

Proportional distribution of error categories across relational UML components in the Seq2AST model.

Component	E1: Structural Sparsity (%)	E2: Semantic Ambiguity (%)	E3: Directional/Label (%)	E4: Multiplicity Error (%)
Dependency	20	38	28	14
Extension	31	24	30	15
Association	28	33	23	16
Aggregation	41	29	17	13
Composition	42	23	20	15

**Fig. 7.** Correctness and completeness of UML elements, and distribution of error categories across relational components.

B extends A instead of A extends B) or produces an incorrect or absent relationship label. This category accounts for a notable proportion of dependency and extension errors, since the source-to-target ordering is often ambiguous in passive or inverted sentence constructions (e.g., “*Functionality is inherited by the subclass from its parent*”).

E4 — Multiplicity errors: The model generates the correct structural relation but assigns incorrect or missing multiplicities (e.g., generating ONE–MANY where ground truth specifies MANY–MANY). Multiplicity errors are relatively consistent across all relational types at approximately 12%–16% of mismatches, suggesting a systematic underrepresentation of multiplicity cues in the training data rather than a type-specific limitation.

4.4.2. Proportional error distribution

Table 6 presents the proportional contribution of each error category to the total mismatches observed per relational component type. The percentages are normalized within each component to sum to 100%, enabling comparison of the relative dominance of each failure mode.

As shown in Fig. 7, structural sparsity errors become increasingly dominant for Aggregation and Composition relationships, accounting for more than 40% of the observed mismatches. In contrast, Dependency and Association relationships exhibit a larger proportion of semantic ambiguity errors. Directional/label errors remain relatively consistent across relationship types, whereas multiplicity errors constitute the smallest error category for all components. These findings suggest that the primary challenge for Seq2AST lies in recovering structurally complex relationship patterns rather than identifying multiplicity constraints.

4.4.3. Analysis of low-scoring components

Aggregation is primarily affected by E1 structural sparsity errors (41%). Since natural language requirements rarely express aggregation explicitly, the model often defaults to the semantically weaker association relation. This issue is further amplified by the relatively low frequency of aggregation examples in the training data, resulting in a weaker learning signal.

Composition exhibits the highest E1 rate (42%), indicating that composition relations are frequently omitted. The strongest whole-part dependency encoded by composition is often implied rather than explicitly stated in requirements, making it difficult to distinguish from aggregation or association. Additionally, composition is among the least represented relation types in the dataset, leading to reduced sensitivity to composition-specific patterns. The low E2 rate suggests that when composition is generated, it is usually classified correctly.

Dependency shows a different error profile. While E1 sparsity is relatively low (20%), E2 semantic ambiguity is the highest among all relation types (38%). Dependency-related verbs such as *uses*, *calls*, and *relies on* are often shared with association semantics, making it difficult for the model to distinguish between these relation types. Consequently, dependency errors stem more from relation-type confusion than from missed relation detection.

4.4.4. Discussion and implications

The performance gap between structural and relational components stems from three factors: (i) structural elements are usually stated explicitly in requirements, whereas relations must be inferred from contextual cues; (ii) structural elements occur more frequently in the training data, providing stronger supervision; and (iii) relation generation requires predicting multiple properties, including type, direction, labels, and multiplicities. Consequently, relational components exhibit higher semantic ambiguity (23%–38%) and structural sparsity (20%–42%) than structural components (Table 6).

These findings suggest several directions for improvement. Data augmentation with explicit part-whole and possession patterns may reduce E1 errors, while relation-disambiguation mechanisms could address E2 ambiguities. Explicit source–target markers may help mitigate E3 directionality errors, and treating multiplicity prediction as a dedicated sub-task could improve E4 performance. Together, these enhancements may strengthen relation extraction and improve overall UML diagram quality.

4.5. Robustness evaluation

A significant challenge in the domain of automated UML generation from natural language is the scarcity of publicly available

Table 7

Perturbation types used in the stress testing framework, with linguistic motivation and examples.

Type	Motivation	Original	Perturbed
Token noise (TN)	Typos and OCR errors in scanned documents	<i>The system shall store user credentials</i>	<i>The systm shall stoer usr credntials</i>
Synonym substitution (SS)	Domain-specific paraphrasing and informal language	<i>A customer places an order</i>	<i>A client submits a purchase request</i>
Structural reordering (SR)	Inverted clause order common in informal writing	<i>Each order contains multiple items</i>	<i>Multiple items are contained in each order</i>
Implicit omission (IO)	Underspecified or elliptical requirements	<i>The product has a name, price, and category</i>	<i>The product has various attributes</i>
Noise injection (NI)	Irrelevant or boilerplate text inserted around requirements	<i>A student can enroll in multiple courses</i>	<i>As noted earlier, for reference purposes, a student can enroll in multiple courses going forward</i>

paired datasets—comprising real-world software requirements alongside their corresponding validated class diagrams. To our knowledge, no standardized benchmark of this kind currently exists in the form of annotated industrial or open-source project data. Prior studies in the field have similarly relied on academic specifications, synthetic requirements, or semi-structured documentation (Chen, Yang, et al., 2023; Herchi & Abdesslem, 2012, 2012; More & Phalnikar, 2012). The absence of such resources reflects a broader challenge: industrial requirements are frequently proprietary, inconsistently formatted, and lack ground-truth UML artifacts verified by domain experts.

To further evaluate robustness, we used the six software requirements introduced in the previous section, for which no publicly available ground-truth class diagrams exist. Rather than constructing artificial reference diagrams — which would reintroduce evaluator subjectivity — we applied these requirements within a *stress testing* framework. This framework evaluates the robustness and consistency of the Seq2AST (Integrated) model when inputs are subjected to controlled perturbations that simulate the noise, ambiguity, and informality commonly encountered in real-world software documentation. This framework evaluates robustness by examining whether the model's outputs remain *stable* and *semantically consistent* under varied perturbation conditions.

4.5.1. Perturbation taxonomy

We define five perturbation categories, each targeting a distinct source of noise that commonly appears in real-world requirement documents. Table 7 summarizes each perturbation type, its linguistic motivation, and a concrete example.

These five perturbation types are not applied uniformly. Each perturbation is applied at three severity levels — *mild* (20% of tokens affected), *moderate* (40%), and *severe* (60%) — yielding a total of $6 \times 5 \times 3 = 90$ perturbed requirement sets. For types TN and NI, severity controls the proportion of tokens modified or augmented. For SS and SR, it controls the proportion of sentences affected. For IO, it controls the proportion of explicit attributes or relationships removed from the requirement text. Algorithm 2 formalizes this process.

4.5.2. Evaluation metrics for stress testing

Since no ground-truth diagrams exist for the six requirement sets, classical recall and precision metrics cannot be computed directly. We therefore adopt two complementary robustness metrics.

Semantic consistency score (SCS) Semantic consistency measures the extent to which a perturbed requirement produces the same UML concepts as the corresponding clean requirement. Specifically, we compare the sets of extracted class names and relationship identifiers from the two generated diagrams using the Jaccard similarity coefficient. Higher SCS values indicate that the model preserves the underlying semantic structure despite input perturbations, as in Eq. (6).

$$SCS = \frac{|C \cap C_\sigma^\tau|}{|C \cup C_\sigma^\tau|} \quad (6)$$

where C and C_σ^τ denote the identifier sets extracted from the clean and perturbed outputs, respectively.

Algorithm. 2 Requirement Perturbation for Stress Testing

```

1: Input: Requirement set  $R = \{r_1, \dots, r_n\}$ , perturbation type  $\tau \in \{TN, SS, SR, IO, NI\}$ , severity  $\sigma \in \{0.2, 0.4, 0.6\}$ 
2: Output: Perturbed requirement set  $R_\sigma^\tau$ 
3: for each requirement  $r \in R$  do
4:   Select a proportion  $\sigma$  of the requirement content to modify
5:   if  $\tau = TN$  then
6:     Introduce spelling and character-level errors
7:   else if  $\tau = SS$  then
8:     Replace selected terms with semantically equivalent synonyms // WordNet
9:   else if  $\tau = SR$  then
10:    Reorder sentence or clause structure while preserving semantics
11:   else if  $\tau = IO$  then
12:    Remove or generalize selected attributes and relationships
13:   else if  $\tau = NI$  then
14:    Insert unrelated boilerplate or contextual text
15:   end if
16:   Store the resulting perturbed requirement
17: end for
18: return All perturbed requirements

```

Element retention rate (ERR) measures the proportion of structural elements preserved after perturbation. It is computed as the fraction of AST nodes in the clean output that are also present in the corresponding perturbed output. Higher ERR values indicate greater preservation of the generated UML structure under noisy inputs, as in Eq. (7).

$$ERR = \frac{|\mathcal{N} \cap \mathcal{N}_\sigma^\tau|}{|\mathcal{N}|} \quad (7)$$

where $\mathcal{N}$ and $\mathcal{N}_\sigma^\tau$ denote the sets of AST elements in the clean and perturbed outputs, respectively.

4.5.3. Stress testing results

Table 8 presents the mean SCS and ERR scores across all six requirement documents for each perturbation type, averaged over the three severity levels. The results reveal several notable patterns. Token noise, structural reordering, and noise injection exhibit the strongest robustness, with token noise achieving the highest SCS (86.4%) and noise injection the highest ERR (84.1%). In contrast, synonym substitution reduces semantic consistency (79.2%), indicating sensitivity to lexical variation in domain terminology.

Implicit omission produces the largest degradation, yielding the lowest SCS (69.6%) and ERR (64.3%). This outcome is expected, as removing attributes, entities, or relationships from the requirements naturally leads to less detailed UML diagrams. Consequently, the reduction reflects information loss rather than model brittleness. Overall, the model achieves a mean SCS of 81.4% and ERR of 77.2%, indicating that most semantic concepts and structural elements are preserved.

Table 8

Stress testing results by perturbation type (mean across mild, moderate, severe severities and all 6 requirement sets).

Perturbation Type	SCS (%)	ERR (%)
Token noise (TN)	86.4	78.6
Synonym substitution (SS)	79.2	79.8
Structural reordering (SR)	85.9	79.2
Implicit omission (IO)	69.6	64.3
Noise injection (NI)	85.9	84.1
Overall mean	81.4	77.2

despite substantial perturbations. These findings suggest that the integrated Seq2AST model remains robust under realistic variations in requirement quality and formulation.

The most common limitation is the underrepresentation of service-layer classes (e.g., `NotificationService`, `TrackingService`), which are often described implicitly rather than explicitly in requirements. This reflects the broader challenge of inferring architectural boundaries from natural-language specifications (Cámara et al., 2023; Chen, Yang, et al., 2023). We acknowledge that robustness testing under noisy requirements does not fully replace evaluation on paired industrial requirements and validated UML diagrams, which remain largely unavailable.

5. Discussions

5.1. Discussion of the findings

The results in Section 4.3 highlight the effectiveness of AST-guided generation. Seq2AST achieves notable gains in precision, recall, and F1-score, underscoring the structural benefits of AST-based representations. Additionally, incorporating domain-specific enhancements — such as attribute-type inference and relation constraints — further improved performance, demonstrating the value of design pattern awareness and structural priors in producing accurate UML class diagrams.

The evaluation results in Table 4 show that the Seq2AST (Integrated) model maintains balanced and competitive performance across all UML components. For classes, attributes, methods, and relationships, precision and recall values remain closely aligned — e.g., class (Rec. 94.16%, Prec. 88.27%), attribute (Rec. 91.97%, Prec. 86.37%), method (Rec. 90.77%, Prec. 85.47%), and relationship (Rec. 90.07%, Prec. 87.06%) — indicating the model neither strongly favors precision nor recall. This reflects a well-rounded prediction capability that minimizes both false positives and false negatives. Although state-of-the-art LLMs such as GPT-4-turbo and DeepSeek-V3 slightly outperform Seq2AST in some categories, the integrated model remains highly competitive, particularly in relationship and overall diagram accuracy (F1: 88.54% and 89.20%, respectively), validating the effectiveness of AST-guided generation.

As observed in Table 5, the values for completeness and correctness are generally close across all models, indicating that the outputs tend to align well with the ground truth without substantial overgeneration or omission. This relative proximity suggests that the models produce class diagram elements that are largely accurate and only slightly incomplete or imprecise. Consequently, both metrics tend to vary together in a consistent pattern, reflecting a balance between precision and recall. This behavior implies that the models focus on generating core, relevant components while minimizing extraneous or spurious elements.

From a sustainable systems perspective, these findings are particularly significant. Software design errors introduced during early modeling stages often propagate throughout the development lifecycle, increasing technical debt, maintenance effort, and overall resource consumption. By improving structural fidelity and reducing hallucinations, the AST-guided approach produces more reliable early-stage design artifacts, thereby lowering the likelihood of costly downstream rework.

In addition, Seq2AST employs a lightweight architecture with approximately 60 million parameters—substantially smaller than billion-parameter general-purpose LLMs. This compact design enables competitive predictive performance while significantly reducing computational requirements and power consumption. The reduced computational footprint, combined with improved structural accuracy, supports more energy-efficient and reproducible AI deployment, making the model well suited for integration into continuous integration pipelines, educational environments, and resource-constrained development settings. Overall, the results suggest that specialized, structure-aware transformer models provide a sustainable alternative to large, general-purpose LLMs for domain-specific engineering tasks. Rather than relying solely on scaling model size, embedding domain constraints and structural priors offers a more resource-conscious and reliable pathway toward AI-assisted software design.

5.2. Practical implications

A key advantage of the proposed Seq2AST approach is its determinism and syntactic correctness. Unlike general-purpose LLMs such as GPT-based models, which rely on probabilistic decoding strategies (e.g., top- k or top- p sampling) and may occasionally produce hallucinated or structurally inconsistent outputs, Seq2AST generates syntactically valid UML representations through deterministic AST parsing. This ensures higher reliability in software modeling, where small structural errors can lead to major design flaws.

Furthermore, Seq2AST's training on domain-specific software engineering data reduces its susceptibility to hallucinations compared to general-purpose LLMs. This highlights the value of structurally grounded, domain-aware architectures for applications that demand interpretability and consistency. It also underscores the need for robust, task-specific benchmarking frameworks that go beyond surface-level generation quality.

Additionally, qualitative evaluation reveals the limitations of token-based metrics such as BLEU, which may unfairly penalize semantically equivalent terms (e.g., `transactionNo` vs. `transactionId`). This suggests the need for complementary evaluation metrics that better reflect structural and semantic correctness in UML generation. Finally, as Seq2AST depends on extracting structural context from natural language requirements, the input text should be sufficiently informative — though not restricted to a fixed length — to support the generation of accurate and complete class diagram elements.

5.3. Framework extension to sequence diagrams

To partially address RQ5, we present a structured proof-of-concept demonstrating how the proposed AST-based framework can be extended to UML sequence diagrams. Unlike class diagrams — which are inherently structural and order-independent — sequence diagrams are *behavioral* and *order-sensitive*, requiring mechanisms to capture temporal dependencies and interaction ordering. This section describes the necessary adaptations to the AST schema, grammar, and inference pipeline, and validates the approach on a small illustrative example.

5.3.1. Temporal challenges in sequence diagram generation

The core challenge in adapting our framework to sequence diagrams lies in the temporal dimension. In class diagrams, elements such as classes, attributes, and relationships are context-independent and can be generated in any order—a key design property of our current AST schema. Sequence diagrams, however, impose strict ordering constraints: messages between lifelines must occur in a defined sequence, conditional fragments (`alt`/`opt`) must nest correctly, and synchronous calls must be paired with their corresponding return messages.

To address this, we introduce an extended AST variant — *S-AST* — that augments the base schema with temporal ordering fields. Each message node carries an explicit order index and an optional predecessor reference, enabling the tree to encode partial orderings that survive traversal in any direction. This design preserves the context-independence property of individual nodes while encoding the global temporal structure through node-level metadata, as illustrated in Schema 2.

Schema. 2 S-AST Schema Structure for Sequence Diagrams

```

Root Node:
  type = "seq_root"
  children: List of {LifelineNode}

LifelineNode:
  type = "lifeline"
  value: String // actor or participant name
  stereotype: String (default "participant")
  children: [] // structural only; messages live in MessageNodes

MessageNode:
  type = "message"
  order: Integer // global sequence index (1-based)
  sender: String // lifeline name
  receiver: String // lifeline name
  label: String
  message_type: String // sync | async | return | create | destroy
  predecessor: Integer (optional) // order index of preceding message
  children: List of {FragmentNode} // nested alt/opt/loop

FragmentNode:
  type = "fragment"
  fragment_type: String // alt | opt | loop | par
  condition: String
  order_start: Integer
  order_end: Integer
  children: List of {MessageNode}

Dependency:
  For every MessageNode with predecessor ≠ null:
    order[predecessor] < order[current] must hold globally.

```

The `order` field encodes absolute temporal position, while the `predecessor` field captures direct causal dependencies—enabling reconstruction of the full temporal chain even when messages are generated non-sequentially by the model. The `FragmentNode` nests a sub-sequence with its own bounded order range (`order_start`, `order_end`), preserving the hierarchy of conditional and loop constructs.

5.3.2. Grammar extension for sequence diagrams

The grammar defined in Fig. 5 is extended with sequence-specific constructs, as shown in Fig. 8. The key additions are the `[ORDER]` token, which precedes each message and establishes its temporal index, and the `[PRED]` token, which links a message to its causal predecessor. Message types are encoded as terminal keywords (`[SYNC]`, `[ASYNC]`, `[RETURN]`), and fragment boundaries use explicit open/close markers (`[FRAG:alt]`/`[/FRAG]`). This grammar remains fully compatible into a *S-AST* and can be subsequently rendered into PlantUML sequence notation.

5.3.3. Running example: Online order processing

To validate the feasibility of this extension, we apply the adapted pipeline to the following software requirement:

“A customer places an order through the web interface. The system validates the order and checks inventory. If stock is available, the payment service processes the payment and the order is confirmed; otherwise, the customer is notified of unavailability.”

Fig. 9 illustrates the full transformation. The *S-AST* captures the five participating lifelines (Customer, WebUI, OrderService, InventoryService, PaymentService), seven ordered messages, and an `alt` fragment encoding the conditional branching on stock availability. The predecessor links form a directed acyclic graph over the messages, ensuring that the temporal chain is recoverable independent of generation order.

The generated diagram correctly captures all seven messages, preserves the temporal ordering (messages 1–3 occur unconditionally; messages 4–7 are conditional on stock availability), and correctly nests the `alt` fragment. The predecessor links in the *S-AST* (e.g., message 4 has `predecessor=3`; message 5 has `predecessor=4`) guarantee that any linearization of the AST respects the original temporal chain.

5.3.4. Multi-task adaptation for sequence diagrams

The multi-task learning pipeline introduced in Section 3.2.2 adapts naturally to sequence diagrams by defining three analogous identification tasks: *Lifeline Identification* (analogous to Class Identification), *Message Identification* (analogous to Attribute and Method Identification), and *Fragment Identification* (analogous to Relation Identification). The key modification lies in the Message Identification task, which must additionally predict the order and predecessor fields. To supervise temporal ordering, we augment the training objective with an auxiliary ordering consistency loss $\mathcal{L}_{\text{ord}}$, which penalizes violations of the constraint $\text{order}[\text{predecessor}] < \text{order}[\text{current}]$ across all generated message pairs:

$$\mathcal{L}_{\text{ord}} = \sum_{(m_i, m_j) \in \mathcal{P}} \max(0, \text{order}(m_i) - \text{order}(m_j) + 1) \quad (8)$$

where $\mathcal{P}$ is the set of all predecessor–message pairs in the training batch. The total training loss combines cross-entropy loss $\mathcal{L}_{\text{CE}}$ with the ordering consistency term:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{ord}} \quad (9)$$

with λ treated as a hyperparameter controlling the trade-off between generation quality and temporal consistency. This formulation ensures that the model learns to produce not only syntactically valid sequence diagram elements but also temporally coherent interaction orderings. During inference, the AST reconstruction algorithm (Algorithm 1) is extended to sort message nodes by their order field after generation and to verify predecessor constraints before emitting PlantUML, flagging any inconsistencies for post-processing.

5.3.5. Limitations and scope

While this proof-of-concept demonstrates the feasibility of extending the AST-guided framework to sequence diagrams, several limitations must be acknowledged. First, a full experimental evaluation — including dataset construction, fine-tuning, and quantitative benchmarking — remains as future work; this section addresses the conceptual and structural feasibility of the extension rather than its empirical performance. Second, sequence diagrams present unique challenges absent in class diagrams: deeply nested combined fragments (e.g., `loop` inside `alt`), reference fragments (`ref`), and co-region parallelism require increasingly complex AST representations that may strain the model’s generation capacity. Third, the ordering consistency loss introduces an additional hyperparameter λ whose tuning may be non-trivial on small datasets. Despite these limitations, the structural analogy between the class diagram pipeline and the proposed sequence diagram extension — from the AST schema and grammar design to the

```

[SEQ:SEG]
|-- lifeline*
| +-- [LIFELINE] stereotype lifeline_name
| |-- stereotype -> [ACTOR] | [PARTICIPANT] | [BOUNDARY] | [CONTROL] | [ENTITY]
| +-- lifeline_name -> PascalCase
|-- message*
| +-- [ORDER] index [MSG] sender receiver message_type label [PRED] pred_index
| |-- index -> Integer (1-based, globally unique)
| |-- sender, receiver -> lifeline_name (PascalCase)
| |-- message_type -> [SYNC] | [ASYN] | [RETURN] | [CREATE] | [DESTROY]
| |-- label -> camelCase | [NOLABEL]
| +-- pred_index -> Integer | [NONE]
|-- fragment*
| +-- [FRAG:type] condition [ORDER_RANGE] start end
| |-- type -> alt | opt | loop | par
| |-- condition -> camelCase | [NOCOND]
| |-- start, end -> Integer (bounding the nested message indices)
| +-- children: message* (same structure as above)
[/SEQ:SEG]

```

Fig. 8. Grammar extension for sequence diagram generation. The [ORDER] and [PRED] tokens encode temporal position and causal dependency, respectively, enabling deterministic reconstruction of interaction ordering from non-sequential model outputs.

multi-task decomposition strategy — demonstrates that the framework can be systematically extended across UML diagram families in a principled manner. This provides evidence of the framework’s architectural generalizability while acknowledging that empirical validation through dataset construction, model training, and quantitative evaluation remains a direction for future work, thereby supporting the broader claim of RQ5.

5.4. Threats to validity

While our results are promising, several limitations and threats to validity must be acknowledged:

5.4.1. Internal validity

While the dataset was constructed via deterministic transformation processes, the manual annotations for correctness and completeness may introduce subjective bias, despite the use of a blind evaluation strategy. However, to quantitatively assess the reliability of the human evaluation, we conducted a post-hoc inter-rater agreement study. We randomly selected 9 diagrams (approximately 25% of the 36-diagram evaluation set). Three independent evaluators rated the same diagrams using the defined Correctness and Completeness formulas (Eqs. (4) and (5)) across all evaluated models. The raters’ continuous scores were discretized into four ordinal categories (Poor: 0–0.40, Moderate: 0.41–0.60, Good: 0.61–0.80, Excellent: 0.81–1.00) to enable computation of agreement statistics on ordinal ratings. Since three raters were involved, we computed Fleiss’ Kappa, which generalizes Cohen’s Kappa to multiple raters. The results showed substantial agreement (Fleiss’ Kappa = 0.78 for Correctness, 0.74 for Completeness; Landis & Koch benchmarks: 0.61–0.80 = substantial). These values confirm that the qualitative evaluation protocol is consistent across independent evaluators, mitigating concerns about subjective bias.

Additionally, LLM-based paraphrased data augmentation may introduce subtle biases due to the probabilistic nature of text generation. To mitigate this threat, paraphrasing was performed using multiple LLMs, reducing dependence on any single model and minimizing generation-specific patterns. Furthermore, evaluation incorporated BLEU, correctness, and completeness metrics to assess both linguistic similarity and structural accuracy, thereby reducing the risk of biased conclusions based solely on surface-level text similarity.

5.4.2. External validity

Our dataset focuses on class diagrams derived from academic literature and industry-based specifications rather than large-scale real-world software projects. Consequently, the generalizability of the proposed approach may be limited when applied to industrial requirements, which often exhibit greater ambiguity, inconsistency, incompleteness, and domain-specific terminology. In addition, the dataset consists exclusively of English-language requirements, limiting conclusions regarding multilingual applicability. To partially mitigate these threats,

the dataset incorporates requirements from diverse application domains and varying levels of complexity. Furthermore, robustness testing with controlled requirement perturbations indicates that the proposed Seq2AST framework remains stable under realistic requirement variations.

The applicability of the proposed framework to other UML diagram types, particularly behavioral diagrams such as sequence diagrams, requires further empirical validation. As discussed in Section 5.3, the framework can be conceptually extended through modifications to the AST schema, grammar, and training objectives to capture temporal dependencies and interaction ordering. However, the proposed extension has not been evaluated on a dedicated sequence diagram dataset, and its effectiveness in real-world settings therefore remains unverified.

5.4.3. Construct validity

We assess model performance using BLEU. However, BLEU may fall short in capturing semantic equivalence—particularly when minor lexical differences, such as variations in identifier naming, are present. From a software engineering perspective, evaluating diagram quality may require more nuanced metrics or human-in-the-loop assessments to better reflect structural and semantic correctness.

A key threat to construct validity lies in the clarity and completeness of the input software requirements, as the model’s performance is highly sensitive to these factors. In practical software projects, generated diagrams may range from fully correct to partially incomplete, depending on the ambiguity or precision of the requirements. Unclear, incomplete, or conflicting inputs can lead to inaccurate diagram generation.

6. Conclusion

This study presented a structure-aware transformer framework for translating natural language software requirements into UML class diagrams through Seq2AST generation and domain-specific structural enhancements. The experimental results demonstrated that embedding AST constraints significantly improves structural fidelity and semantic consistency. The enhanced Seq2AST model achieved an F1-score of 89.91%, substantially outperforming traditional Seq2Seq approaches (F1: 71.02%), thereby confirming that structural priors are critical for preserving UML semantics during generation. Component-wise refinements further increased Recall and Precision by 3.69% and 5.51%, respectively. While zero-shot LLMs such as GPT-4-turbo achieved marginally higher correctness scores (81.6% vs. 78.2%), Seq2AST produced more deterministic and reproducible outputs with fewer hallucinations, making it better suited for reliability-sensitive and engineering-critical contexts.

Beyond performance gains, this work contributes to the broader goal of sustainable AI in software engineering. By employing a lightweight architecture with approximately 60 million parameters — significantly smaller than billion-parameter general-purpose LLMs — the proposed

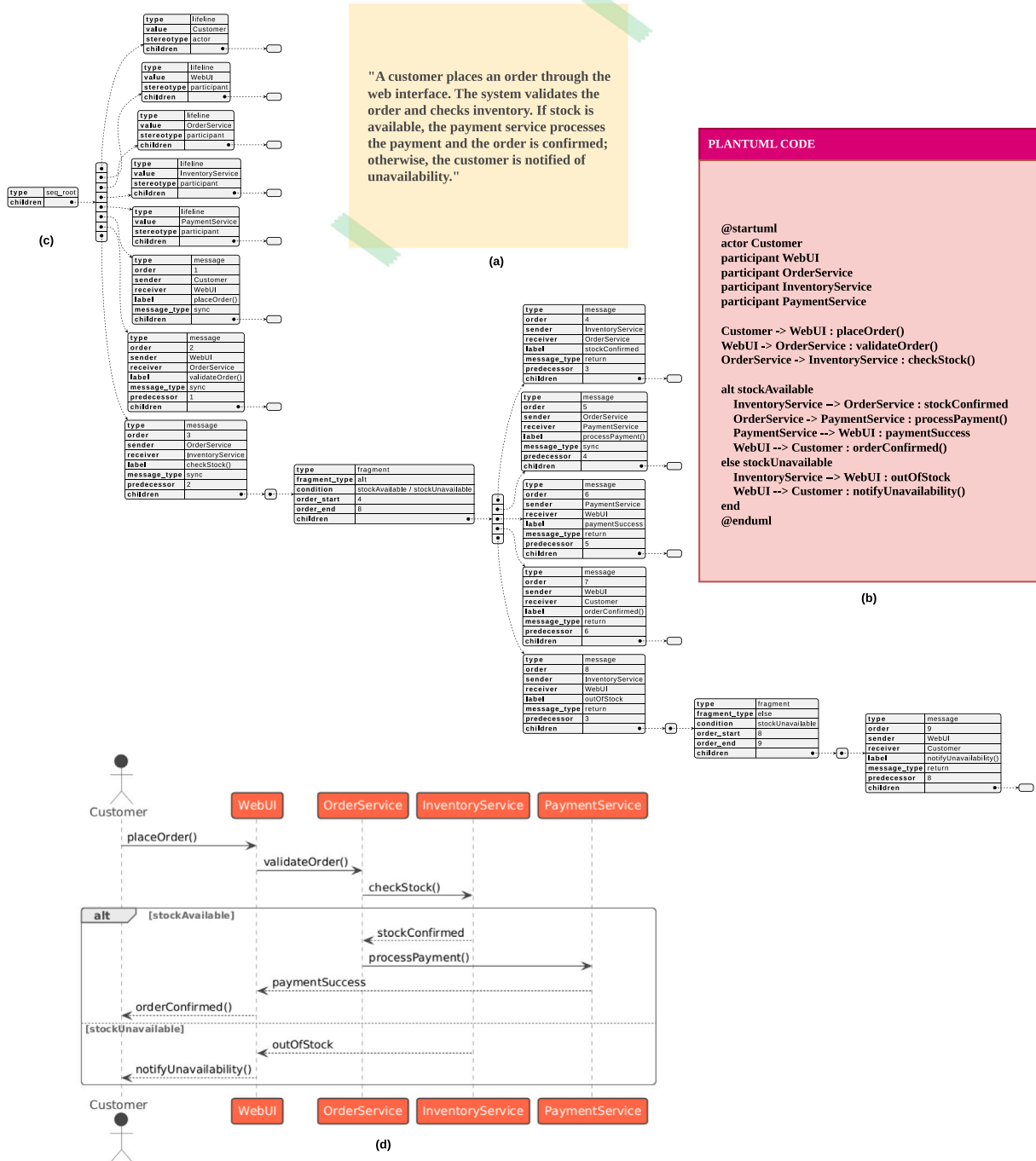


Fig. 9. Transformation pipeline for sequence diagram generation: (a) natural language requirement; (b) generated PlantUML code; (c) S-AST with lifelines, ordered messages, and a conditional alt fragment; (d) rendered UML sequence diagram. The order indices and predecessor links in the S-AST encode temporal dependencies, ensuring correct interaction ordering independent of generation sequence.

approach achieves competitive accuracy with a substantially reduced computational footprint. This reduction in model scale lowers energy consumption while maintaining structural precision, enabling more resource-efficient AI deployment. Importantly, the compact design allows the model to be packaged as a lightweight plugin directly integrated into Integrated Development Environments (IDEs), rather than relying on external, cloud-hosted LLM services. Such local deployment reduces latency, minimizes infrastructure dependency, enhances data privacy, and avoids the recurring computational and environmental

costs associated with large-scale model inference. Consequently, the approach supports practical, real-time AI assistance within development workflows while adhering to principles of sustainable and responsible software engineering.

Although the framework can be extended to other UML diagram types, particularly behavioral diagrams such as sequence diagrams, such extensions require explicit modeling of temporal and behavioral dependencies, as demonstrated in our proof-of-concept analysis. To foster reproducibility and continued advancement, we publicly release

our datasets, training scripts, and model checkpoints, enabling the research community to replicate and build upon our findings. Future work includes extending the approach to other behavioral diagrams (e.g., activity diagrams), integrating human-in-the-loop feedback mechanisms for iterative refinement, and developing evaluation metrics that better capture semantic equivalence beyond surface-level similarity measures such as BLEU.

This work bridges natural language processing and automated software modeling by demonstrating that domain-constrained, structure-aware transformer models provide a sustainable and reliable alternative to large, general-purpose LLMs. Rather than relying solely on model scaling, embedding structural priors offers a more energy-conscious, interpretable, and engineering-aligned pathway toward AI-assisted software design.

CRedit authorship contribution statement

Feisal Alaswad: Conceptualization, Methodology, Investigation, Writing – Original Draft. **Poovammal E.:** Supervision, Validation, Writing – Review & Editing. **Batoul Aljaddouh:** Data Curation, Formal Analysis, Writing – Review & Editing. **Kadiyala Ramana:** Resources, Investigation. **Arfat Ahmad Khan:** Methodology, Writing – Review & Editing. **Natalia Kryvinska:** Methodology, Writing – Review & Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix. Software requirements and data augmentation

To improve model robustness and generalization, we applied data augmentation by paraphrasing software requirements and updating the corresponding UML diagrams accordingly. The example below illustrates a typical password reset process via email, modeled using the User and PasswordResetService classes.

Original requirement and UML representation

The system shall allow users to reset their password via email verification. A unique, time-limited link should be sent to the registered email address upon request.

The original UML model, shown in Listing 1, outlines the reset process using the User and PasswordResetService classes.

Listing 1: Original UML Representation

```
@startuml
class User {
- email: String
- password: String
+ requestPasswordReset(): void
+ resetPassword(token: String, newPassword: String): void
}

class PasswordResetService {
+ generateResetToken(email: String): String
+ sendResetLink(email: String, token: String): void
+ validateToken(token: String): boolean
}

User --> PasswordResetService : uses
@enduml
```

Paraphrased version for augmentation

The paraphrased version maintains the core semantics and structural alignment while introducing minor lexical variations to support robustness without diverging in meaning or design.

Users should be able to change their password using an email-based verification process. When a reset is requested, a unique and time-bound link will be emailed to the user's registered address.

The paraphrased UML model is shown in Listing 2. It reflects the same behavior with alternative wording and renamed method identifiers.

Listing 2: Paraphrased UML Representation

```
@startuml
class User {
- email: String
- password: String
+ initiatePasswordReset(): void
+ updatePassword(token: String, newPassword: String): void
}

class PasswordResetService {
+ createResetToken(email: String): String
+ emailResetLink(email: String, token: String): void
+ checkTokenValidity(token: String): boolean
}

User --> PasswordResetService : interacts with
@enduml
```

Data availability

The code and dataset are made available at: <https://github.com/FeisalAlaswad/GenClass-Seq2AST> and <https://github.com/FeisalAlaswad/PlantUCD-dataset-full>.

References

- Abdelnabi, E. A., Maatuk, A. M., & Hagal, M. (2021). Generating uml class diagram from natural language requirements: A survey of approaches and techniques. In *2021 IEEE 1st international Maghreb meeting of the conference on sciences and techniques of automatic control and computer engineering MI-STA* (pp. 288–293). IEEE, <http://dx.doi.org/10.1109/MI-STA52233.2021.9464433>.
- Ahmed, S., Ahmed, A., & Eisty, N. U. (2022). Automatic transformation of natural to unified modeling language: A systematic review. In *2022 IEEE/aCIS 20th international conference on software engineering research, management and applications* (pp. 112–119). IEEE, <http://dx.doi.org/10.1109/SERA54885.2022.9806783>.
- Alaswad, F. (2025). GitHub – FeisalAlaswad/PlantUCD-dataset-full: a comprehensive collection of plantuml code paired with corresponding software requirements. <https://github.com/FeisalAlaswad/PlantUCD-dataset-full>. Dataset.
- Alaswad, F., Poovammal, E., & Aljaddouh, B. (2026a). From cocomo to gpt: A comprehensive evaluation of llm-based software effort estimation. *IEEE Access*, <http://dx.doi.org/10.1109/ACCESS.2026.3671204>.
- Alaswad, F., Poovammal, E., & Aljaddouh, B. (2026b). Plantucd: A dataset of software requirements and corresponding plantuml-based class diagrams. <http://dx.doi.org/10.36227/techrxiv.176772833.35984408/v1>, Authorea.
- Alaswad, F., Poovammal, E., Aljaddouh, B., & Supriya, B. (2023). Software requirements to UML class diagrams using machine learning and rule-based approach. In *International conference on intelligent systems in computing and communication* (pp. 86–101). Springer, http://dx.doi.org/10.1007/978-3-031-75605-4_6.
- Almazroi, A. A., Abualigah, L., Alqarni, M. A., Houssein, E. H., AlHamad, A. Q. M., & Elaziz, M. A. (2022). Class diagram generation from text requirements: An application of natural language processing. In *Deep learning approaches for spoken and natural language processing* (pp. 55–79). Springer, http://dx.doi.org/10.1007/978-3-030-79778-2_4.
- Amune, A., & Naik, R. (2016). Reflecting natural language text in to UML diagrams. *Spryran's International Journal of Engineering Sciences & Technology (SEST)*, URL: <https://api.semanticscholar.org/CorpusID:3617185>.
- Antoniou, C., & Bassiliades, N. (2024). A tool for requirements engineering using ontologies and boilerplates. *Automated Software Engineering*, 31, 5. <http://dx.doi.org/10.1007/s10515-023-00403-y>.
- Asim, M. N., Wasim, M., Khan, M. U. G., Mahmood, W., & Abbasi, H. M. (2018). A survey of ontology learning techniques and applications. *Database*, 2018, bay101. <http://dx.doi.org/10.1093/database/bay101>.

- Atlanmod (2020). GitHub - atlanmod/atlanmod-zoo. Retrieved from: <https://github.com/atlanmod/atlanmod-zoo>. (Accessed 22 October 2024).
- Bashir, N., Bilal, M., Liaqat, M., Marjani, M., Malik, N., & Ali, M. (2021). Modeling class diagram using nlp in object-oriented designing. In *2021 national computing colleges conference* (pp. 1–6). IEEE, <http://dx.doi.org/10.1109/NCCC49330.2021.9428817>.
- Budake, R., Bhoite, S., & Kharade, K. (2023). A study of AI-based techniques for requirement analysis in software engineering. In *AIP conference proceedings: Vol. 2946*, AIP Publishing LLC, Article 050007. <http://dx.doi.org/10.1063/5.0178114>.
- Cámara, J., Burgueño, L., & Troya, J. (2024). Towards standardized benchmarks of LLMs in software modeling tasks: a conceptual framework. *Software and Systems Modeling*, 23, 1309–1318. <http://dx.doi.org/10.1007/s10270-024-01206-9>.
- Cámara, J., Troya, J., Burgueño, L., & Vallecillo, A. (2023). On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. *Software and Systems Modeling*, 22, 781–793. <http://dx.doi.org/10.1007/s10270-023-01105-5>.
- Chaaben, M. B., Burgueño, L., & Sahraoui, H. (2023). Towards using few-shot prompt learning for automating model completion. In *2023 IEEE/ACM 45th international conference on software engineering: New ideas and emerging results* (pp. 7–12). <http://dx.doi.org/10.1109/ICSE-NIER58687.2023.00008>.
- Chen, K., Yang, Y., Chen, B., Lopez, J. A. H., Mussbacher, G., & Varro, D. (2023). Automated domain modeling with large language models: A comparative study. In *2023 ACM/IEEE 26th international conference on model driven engineering languages and systems* (pp. 162–172). IEEE, <http://dx.doi.org/10.1109/MODELS58315.2023.00037>.
- Chen, B., Yi, F., & Varró, D. (2023). Prompting or fine-tuning? A comparative study of large language models for taxonomy construction. In *2023 ACM/IEEE international conference on model driven engineering languages and systems companion* (pp. 588–596). IEEE, <http://dx.doi.org/10.1109/MODELS-C59198.2023.00097>.
- Ferrari, A., Abualhaija, S., & Arora, C. (2024). Model generation with LLMs: From requirements to UML sequence diagrams. In *2024 IEEE 32nd international requirements engineering conference workshops* (pp. 291–300). IEEE, <http://dx.doi.org/10.1109/REW61692.2024.00044>.
- Gerosa, M., Trinkenreich, B., Steinmacher, I., & Sarma, A. (2024). Can AI serve as a substitute for human subjects in software engineering research? *Automated Software Engineering*, 31, 13. <http://dx.doi.org/10.1007/s10515-023-00409-6>.
- Gruber, T. R. (1995). Toward principles for the design of ontologies used for knowledge sharing? *International Journal of Human-Computer Studies*, 43, 907–928. <http://dx.doi.org/10.1006/ijhc.1995.1081>.
- Herchi, H., & Abdessalem, W. B. (2012). From user requirements to UML class diagram. <http://dx.doi.org/10.48550/arXiv.1211.0713>, arXiv preprint arXiv:1211.0713.
- Javed, M., & Lin, Y. (2018). Iterative process for generating ER diagram from unrestricted requirements. In *Proceedings of the 13th international conference on evaluation of novel approaches to software engineering* (pp. 192–204). Setubal, PRT: SCITEPRESS - Science and Technology Publications, Lda, <http://dx.doi.org/10.5220/0006778701920204>.
- Kusupati, U., & Ailavarapu, V. R. T. (2022). Natural language to code using transformers. <http://dx.doi.org/10.48550/arXiv.2202.00367>, arXiv preprint arXiv:2202.00367.
- Li, X., Qu, Y., & Yin, H. (2021). Palmtree: Learning an assembly language model for instruction embedding. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security* (pp. 3236–3251). <http://dx.doi.org/10.1145/3460120.348458>.
- López, J. A. H., Canovas Izquierdo, J. L., & Cuadrado, J. S. (2022). ModelSet: a dataset for machine learning in model-driven engineering. *Software and Systems Modeling*, 21, 967–986. <http://dx.doi.org/10.1007/s10270-021-00929-3>.
- Lucassen, G., Robeer, M., Dalpiaz, F., Van Der Werf, J. M. E., & Brinkkemper, S. (2017). Extracting conceptual models from user stories with visual narrator. *Requirements Engineering*, 22, 339–358. <http://dx.doi.org/10.1007/s00766-017-0270-1>.
- More, P., & Phalnikar, R. (2012). Generating UML diagrams from natural language specifications. *International Journal of Applied Information Systems*, 1, 19–23. <http://dx.doi.org/10.5120/ijais12-450222>.
- Nikiforova, O., Sejans, J., & Cernickins, A. (2011). Role of UML class diagram in object-oriented software development. *Scientific Journal of Riga Technical University, Series Computer Sciences*, 44, 65–74. <http://dx.doi.org/10.2478/v10143-011-0023-4>.
- Papineni, K., Roukos, S., Ward, T., & Zhu, W. -J. (2002). Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the association for computational linguistics* (pp. 311–318). <http://dx.doi.org/10.3115/1073083.1073135>.
- Roques, A. (2015). Plantuml: Open-source tool that uses simple textual descriptions to draw uml diagrams. URL: <https://plantuml.com>. [Computer software].
- Rush, A. M., Chopra, S., & Weston, J. (2015). A neural attention model for abstractive sentence summarization. In *Proceedings of the 2015 conference on empirical methods in natural language processing* (pp. 379–389). <http://dx.doi.org/10.18653/v1/D15-1044>.
- Soliman, A. S., Hadhoud, M. M., & Shaheen, S. I. (2022). Marianc: a code generation transformer model inspired by machine translation. *Journal of Engineering and Applied Science*, 69, 104. <http://dx.doi.org/10.1186/s44147-022-00159-4>.
- Songyang-dev (2022). GitHub - songyang-dev/uml-classes-and-specs: Repository that contains the data used for 'Extraction of UML Class Diagrams from Natural Language Specification' (Yang et al. 2022). Retrieved from: <https://github.com/songyang-dev/uml-classes-and-specs>. (Accessed 22 October 2024).
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. In *Proceedings of the 28th international conference on neural information processing systems - Volume 2* (pp. 3104–3112). Cambridge, MA, USA: MIT Press.
- Vinyals, O., & Le, Q. (2015). A neural conversational model. <http://dx.doi.org/10.48550/arXiv.1506.05869>, arXiv preprint arXiv:1506.05869.
- Zhong, S., Scarinci, A., & Cicirello, A. (2023). Natural language processing for systems engineering: Automatic generation of systems modelling language diagrams. *Knowledge-Based Systems*, 259, Article 110071. <http://dx.doi.org/10.1016/j.knosys.2022.110071>.
- Zhong, L., Wu, J., Li, Q., Peng, H., & Wu, X. (2023). A comprehensive survey on automatic knowledge graph construction. *ACM Computing Surveys*, 56, 1–62. <http://dx.doi.org/10.1145/3618295>.